FINAL YEAR PROJECT REPORT

AutoPark

A Camera-Based Smart Parking System with Automated Number Plate Recognition and Micro-Actuated Boom Barriers

Submitted by	[Team Member 1] — [USN]
	[Team Member 2] — [USN]
	[Team Member 3] — [USN]
	[Team Member 4] — [USN]
Under the guidance of	[Project Guide]

Department of Computer Science and Engineering
[Institution Name]
[City]

[Month Year]

Declaration

I/We declare that the project report entitled **“AutoPark: A Camera-Based Smart Parking System with Automated Number Plate Recognition and Micro-Actuated Boom Barriers”** submitted to [Institution Name] in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering is a record of original work carried out under the supervision of [Project Guide]. The work has not been submitted, either in part or in full, for the award of any other degree or diploma of this or any other institution.

Where the work of others has been used, it has been duly acknowledged and cited in the reference list.

_____ _____

Place: [City] Signature of the candidate(s)

Date:

Certificate

This is to certify that the project report entitled **“AutoPark: A Camera-Based Smart Parking System with Automated Number Plate Recognition and Micro-Actuated Boom Barriers”** is a bonafide record of the project work carried out by the candidates named below, under my supervision, in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in the Department of Computer Science and Engineering of [Institution Name] during the academic year [2025–26].

[Team Member 1]	[USN]
[Team Member 2]	[USN]
[Team Member 3]	[USN]
[Team Member 4]	[USN]

_____

Project Guide
[Project Guide]

_____

Head of the Department

_____

Internal Examiner

_____

External Examiner

Acknowledgement

We record our sincere gratitude to our project guide, [Project Guide], for the direction, review and patient criticism that shaped this work from an idea into a working system. We thank the Head of the Department of Computer Science and Engineering, [Institution Name], for the facilities and the freedom to build and test the prototype on the department premises.

We thank the laboratory staff for access to the electronics bench, the measurement equipment and the network used during the hardware trials, and our classmates who acted as test drivers and whose unreadable plates, mistyped bookings and determined attempts to break the gate produced most of the defect log in Chapter 6.

Finally, we thank our families for their support through the semesters in which this project competed with everything else.

— The Authors

Abstract

Finding a free parking space in dense commercial and campus environments is slow, wasteful and difficult to monitor. Conventional "smart" parking installations solve the visibility problem by instrumenting every bay with an ultrasonic, magnetic or infrared sensor, which multiplies hardware cost, cabling and maintenance with lot size. This project presents AutoPark, a smart parking system that replaces per-bay sensing with **two camera nodes at the entry and exit gates**. Each node captures a frame every five seconds, transmits it to a FastAPI backend, and the backend locates the licence plate with a YOLOv11 detector and reads it with EasyOCR using four preprocessing variants and multi-frame voting. The system supports both guaranteed advance pre-bookings and unreserved walk-in entry: a pre-booked vehicle is routed to its held bay, while a walk-in vehicle is automatically allocated the next available bay, raising the SG90 boom barrier and highlighting the assigned bay on the live dashboard. If the lot is full or access is ambiguous, the barrier remains closed. Exit is handled symmetrically: the plate is matched against active sessions, the bay is released, and an itemised receipt is produced. Because occupancy is derived from entry and exit events rather than measured per bay, the system's hardware cost is constant with respect to lot size. The implementation comprises a six-bay facility, a real-time Next.js dashboard, a role-aware operator console, a monochrome design system, and a hardware-in-the-loop simulator for demonstration without physical cameras.

Keywords: smart parking, automatic number plate recognition, YOLOv11, EasyOCR, edge IoT, ESP32-CAM, boom barrier, FastAPI, real-time systems.

Table of Contents

1 Introduction 15

1.1 Problem statement 15

1.2 Motivation 15

1.3 Aim 15

1.4 Objectives 16

1.5 Scope 16

1.6 Proposed solution 17

1.7 Novelty and contribution 17

1.8 Stakeholders and personas 17

1.9 Organisation of the report 18

2 Related Work 19

2.1 Occupancy sensing in smart parking 19

2.2 Automatic number plate recognition 19

2.3 Low-cost edge vision and actuation 20

2.4 Real-time monitoring interfaces 21

2.5 Access control, identity and audit 21

2.6 Research gap 21

2.7 Comparison with related approaches 22

3 Requirements Analysis 23

3.1 Actors 23

3.2 Functional requirements 23

3.3 Non-functional requirements 25

3.4 Constraints 25

3.5 Feasibility 26

3.6 Scope boundary 26

4 System Design 27

4.1 Architectural style 27

4.2 Context diagram 27

4.3 Container / component view 28

4.4 End-to-end data flow 29

4.5 State machines 32

4.6 Deployment topology 34

4.7 Database schema 35

4.8 API reference 35

4.9 Pipeline overview 37

4.10 Stage 7 — Database matching 38

4.11 Stage 8 — Policy layer (allocation) 39

4.12 Failure taxonomy 40

4.13 Pin map (both nodes) 40

4.14 Wiring table — Entry node 41

4.15 Wiring table — Exit node 41

4.16 Barrier control contract 41

4.17 Why two cameras instead of per-bay sensors 42

4.18 Design decisions 43

4.19 Architectural invariants 43

4.20 Failure and degradation design 44

5 Implementation 45

5.1 Stack at a glance 45

5.2 Version selection 45

5.3 Runtime topology and practical constraints 46

5.4 Module map 46

5.5 Entry allocation algorithm 47

5.6 Exit release algorithm 47

5.7 Billing . . . 47
5.8 Authentication and security . . . 48
5.9 Concurrency and consistency . . . 49
5.10 WebSocket contract . . . 49
5.11 Startup, migration and reset . . . 49
5.12 Stage 1 — Capture (edge) . . . 50
5.13 Stage 2 — Image quality gate . . . 50
5.14 Stage 3 — Plate localisation (YOLOv11) . . . 51
5.15 Stage 4 — Preprocessing variants . . . 51
5.16 Stage 5 — OCR and candidate scoring . . . 51
5.17 Stage 6 — Multi-frame burst voting . . . 52
5.18 Complexity and performance notes . . . 52
5.19 Purpose of the frontend . . . 53
5.20 Route map . . . 53
5.21 Page behaviour . . . 53
5.22 Design system . . . 55
5.23 Component inventory . . . 55
5.24 Realtime behaviour . . . 56
5.25 Responsiveness and accessibility . . . 56
5.26 Frontend–backend contract . . . 56
5.27 Firmware lifecycle . . . 57

6 Testing and Results . . . 59

6.1 Test strategy . . . 59
6.2 Environment . . . 59
6.3 Automated system tests (executed) . . . 59
6.4 Manual / UI checks . . . 61
6.5 Hardware-in-the-loop checks . . . 61
6.6 Defect log . . . 62

6.7 Regression checklist . . . 62
6.8 Verified functional results . . . 63
6.9 Measurement definitions and protocol . . . 63
6.10 System-level evaluation . . . 65
6.11 Threats to validity . . . 66

7 Conclusion and Future Work . . . 67
7.1 Summary of the work . . . 67
7.2 Objectives achieved . . . 67
7.3 Contribution . . . 68
7.4 Current limitations . . . 69
7.5 Future work . . . 70
7.6 Concluding remarks . . . 72

A Bill of Materials . . . 73

B Assembly, Bench Checklist and Troubleshooting . . . 74
B.1 Assembly procedure . . . 74
B.2 Bench checklist (run before every demo) . . . 74
B.3 Troubleshooting . . . 75
B.4 Hardware limitations . . . 75

C References . . . 76

List of Figures

Figure 4.1 Context diagram 27

Figure 4.2 Container / component view 28

Figure 4.3 Booking (before arrival) 29

Figure 4.4 Entry (authorised) 30

Figure 4.5 Exit 31

Figure 4.6 Parking bay 32

Figure 4.7 Booking 33

Figure 4.8 Session 33

Figure 4.9 Deployment topology 34

Figure 4.10 Database schema 35

Figure 4.11 Capture and reading (stages 1–4) 37

Figure 4.12 Decision path (stages 5–8) 38

Figure 5.1 Firmware lifecycle 57

List of Tables

Table 1.1 Objectives . . . 16

Table 1.2 Out of scope (and why) . . . 16

Table 1.3 Stakeholders and personas . . . 17

Table 2.1 Comparison with related approaches . . . 22

Table 3.1 Actors . . . 23

Table 3.2 Functional requirements . . . 24

Table 3.3 Non-functional requirements . . . 25

Table 3.4 Constraints . . . 25

Table 4.1 Realtime fan-out . . . 32

Table 4.2 System . . . 36

Table 4.3 Detection (edge cameras) . . . 36

Table 4.4 Authentication . . . 36

Table 4.5 Bookings . . . 36

Table 4.6 Parking state and control . . . 37

Table 4.7 Diagnostics . . . 37

Table 4.8 WebSocket . . . 37

Table 4.9 Where each stage lives . . . 38

Table 4.10 Similarity scoring . . . 38

Table 4.11 Acceptance and ambiguity . . . 39

Table 4.12 Failure taxonomy . . . 40

Table 4.13 Pin map (both nodes) . . . 40

Table 4.14 Wiring table — Entry node . . . 41

Table 4.15 Wiring table — Exit node . . . 41

Table 4.16 Truth table . . . 42

Table 4.17 Why two cameras instead of per-bay sensors . . . 42

Table 4.18 Design decisions . . . 43
Table 4.19 Failure and degradation design . . . 44
Table 5.1 Stack at a glance . . . 45
Table 5.2 Runtime topology and practical constraints . . . 46
Table 5.3 Module map . . . 46
Table 5.4 Billing . . . 48
Table 5.5 Authentication and security . . . 48
Table 5.6 Concurrency and consistency . . . 49
Table 5.7 WebSocket contract . . . 49
Table 5.8 Stage 1 — Capture (edge) . . . 50
Table 5.9 Stage 2 — Image quality gate . . . 50
Table 5.10 Stage 3 — Plate localisation (YOLOv11) . . . 51
Table 5.11 Stage 4 — Preprocessing variants . . . 51
Table 5.12 Stage 6 — Multi-frame burst voting . . . 52
Table 5.13 Route map . . . 53
Table 5.14 Operator Console (/operator) . . . 54
Table 5.15 Core tokens (tailwind.config.js) . . . 55
Table 5.16 Component inventory . . . 55
Table 5.17 Firmware lifecycle . . . 58
Table 6.1 Test strategy . . . 59
Table 6.2 Environment . . . 59
Table 6.3 Test case detail . . . 60
Table 6.4 Manual / UI checks . . . 61
Table 6.5 Hardware-in-the-loop checks . . . 61
Table 6.6 Defect log . . . 62
Table 6.7 Functional results (verified) . . . 63

Table 6.8 Engineering metrics (measured from the repository) 63

Table 6.9 Metrics that must still be measured 64

Table 6.10 Recognition performance by condition 65

Table 6.11 Decision outcomes 65

Table 6.12 Latency 65

Table 6.13 Hardware scaling 65

Table 6.14 Threats to validity 66

Table 7.1 Objectives achieved 68

Table 7.2 L6 — Security hardening incomplete 69

Table 7.3 Recognition improvements 71

Table A.1 Bill of Materials 73

Table B.1 Bench checklist (run before every demo) 74

Table B.2 Troubleshooting 75

Abbreviations

ANPR / ALPR	Automatic Number Plate Recognition / Automatic Licence Plate Recognition	MVP	Minimum Viable Product
API	Application Programming Interface	OCR	Optical Character Recognition
CORS	Cross-Origin Resource Sharing	ORM	Object–Relational Mapping
CRUD	Create, Read, Update, Delete	PDR	Plate Detection Rate
CV	Computer Vision	PSRAM	Pseudo-Static Random Access Memory
DB	Database	PWM	Pulse-Width Modulation
ER	Entity–Relationship	REST	Representational State Transfer
GPIO	General-Purpose Input/Output	RF	Random Forest (not used here)
HIL	Hardware-in-the-Loop	SQL	Structured Query Language
HS256	HMAC with SHA-256 (JWT signing algorithm)	TLS	Transport Layer Security
HTTP	Hypertext Transfer Protocol	UI / UX	User Interface / User Experience
IoT	Internet of Things	UTC	Coordinated Universal Time
JPEG	Joint Photographic Experts Group (image format)	VGA / QVGA	640×480 / 320×240 resolution
JSON	JavaScript Object Notation	WS	WebSocket
JWT	JSON Web Token	YOLO	You Only Look Once (object detection family)
LAN	Local Area Network		

CHAPTER 1

Introduction

1.1 Problem statement

Parking search is a systems problem, not a space problem. Drivers cannot see which bays are free before arriving, operators cannot see occupancy without walking the lot, and mechanical or manned gates add recurring labour cost. The two common remedies are both unsatisfactory at small and medium scale. Per-bay sensors give accurate occupancy but require one device, one power drop and one mounting point per bay, so cost grows linearly with capacity. Fully manual gates are cheap to install but cannot provide live availability, audit trails or automated billing.

There is therefore a gap for a system that provides **live bay availability, automated access control and auditable billing** with hardware that does **not scale with the number of bays**.

1.2 Motivation

- **Driver experience.** Arriving without knowing availability causes circling, delay and congestion. Parking-search traffic has been estimated at roughly 30 % of traffic in congested commercial districts [2].
- **Operator economics.** Every additional bay in a sensor-per-bay design adds a device, a cable run and a failure point. A gate-only design adds **zero** marginal sensing hardware per bay.
- **Accountability.** A plate-indexed audit trail links every entry, exit and charge to a vehicle, which manned gates cannot produce.
- **Practicality for a campus.** A six-bay pilot is representative of a department lot, is buildable with hobby-grade hardware, and exercises every part of the real system: authorisation, actuation, billing and monitoring.

1.3 Aim

To design, implement and validate a smart parking system in which **occupancy is inferred from gate events rather than measured per bay**, while preserving accurate availability, controlled access and auditable billing.

1.4 Objectives

Table 1.1 — Objectives

#	Objective	How it is met in the build	How it is evidenced
O1	Detect and read vehicle plates from gate frames	YOLOv11 plate detector + EasyOCR with 4 preprocessing variants and 3-frame voting	Working pipeline; per-stage diagnostics API
O2	Authorise pre-booked and walk-in vehicles	Unique match against `PENDING_ARRIVAL` bookings or dynamic free bay assignment via `ALLOW_WALKIN_ENTRY`	Simulated walk-in & booking tests; full lot refusal
O3	Honour the bay the driver booked	Booking binds `slot_id`; entry allocator prefers that bay	Executed test: B3 requested → B3 allocated
O4	Derive occupancy from gate events	Slot state machine driven by entry/exit only	Layout, summary and WebSocket events consistent after entry/exit
O5	Control a physical barrier automatically	SG90 servo, 0°/90°, 5 s hold, gated on backend `open_barrier`	Firmware logic + API contract verified
O6	Provide real-time visibility	WebSocket broadcast to dashboard, pass and operator console	Live updates without refresh
O7	Produce auditable billing	Duration computed at exit; ₹30 first hour, +₹20 per started hour	Receipts page and exit response verified
O8	Allow demonstration without hardware	Hardware-in-the-loop simulator calling the same endpoints	Authorised / unauthorised / exit / reset flows

1.5 Scope

1.5.1 In scope

- One parking facility with six bays across two zones.
- Two gate camera nodes (entry, exit) with servo barriers.
- Pre-booking through a web application with bay selection.
- Dynamic walk-in entry allocation for spontaneous arrivals.
- Plate-based entry authorisation and exit release.
- Real-time dashboard, driver pass, receipts and operator console.
- Billing computation and receipt generation.
- Simulation mode for demonstration.

1.5.2 Out of scope (and why)

Table 1.2 — Out of scope (and why)

Excluded	Reason
Online payment	Adds a payment-gateway integration and compliance surface with no bearing on the core contribution. Billing is computed and itemised, not charged.
Multi-level / multi-lot management	The architecture generalises, but a second lot multiplies hardware and is unnecessary to demonstrate the concept.
Native mobile app	The web app is responsive and installable in a browser; a native client is a packaging exercise.
Per-bay sensors	Excluded by design — this is the hypothesis under test.

1.6 Proposed solution

```
Driver books a bay  →  Booking locks the bay (ALLOCATED)
        ↓
Car reaches entry gate  →  ESP32-CAM posts a frame
        ↓
YOLO locates plate  →  EasyOCR reads it  →  burst voting  →  fuzzy match
        ↓                                                       ↓
unique booking                                        no / ambiguous match
        ↓                                                       ↓
bay bound · session ACTIVE · servo 90°                barrier stays 0° · alert
        ↓
Driver sees "BAY A2" on /pass
        ↓
Car reaches exit gate  →  plate matched to ACTIVE session
        ↓
bay released · bill computed · servo 90° · receipt
```

> **A bay is free if and only if a vehicle has entered and not yet left — the database, not a sensor, is the source of occupancy truth.**

1.7 Novelty and contribution

1. **Gate-only occupancy inference for a multi-bay lot.** Occupancy is a derived state over entry/exit events, so sensing hardware is O(1) in the number of bays instead of O(n). Most student and commercial systems instrument each bay.
2. **Refusal-by-default authorisation.** The system is designed to *deny* when evidence is insufficient. Low confidence, short readings, ambiguous matches and unknown plates all leave the barrier closed. This inverts the usual "best effort admit" behaviour of demo ANPR systems.
3. **OCR-aware plate matching.** Canonicalisation of visually confusable characters (O/0, I/1, S/5, B/8, Z/2, G/6) combined with Levenshtein distance and a confidence-weighted score lets `KA05KK0403` match `KA05KK04O3` without merging two genuinely different plates.
4. **End-to-end integration.** Detection, authorisation, actuation, billing, realtime UI and a hardware simulator are one coherent system rather than disconnected demos.

1.8 Stakeholders and personas

Table 1.3 — Stakeholders and personas

Persona	Need	System response
Driver	Know a bay is waiting; get in without talking to anyone	Pre-book, guaranteed bay, pass screen, barrier opens on plate match
Operator / security	See the lot, handle exceptions, release stuck bays	Operator console: live bays, camera status, diagnostics, manual release, reset
Facility owner	Lower hardware cost, audit trail, revenue record	Two cameras regardless of bay count; per-session receipts; detection event log
Project evaluator	Verify claims, see failure behaviour	Diagnostics endpoint, evidence capture, explicit refusal paths, simulation controls

1.9 Organisation of the report

The remainder of this report follows the conventional structure of an engineering dissertation. Chapter 2 reviews the three bodies of work that AutoPark builds on — occupancy sensing, number plate recognition and gate actuation — and states the gap. Chapter 3 converts the problem into functional and non-functional requirements and examines feasibility. Chapter 4 presents the design: the layered architecture, the data-flow and state models, the database and API contracts, the recognition pipeline and the hardware design. Chapter 5 describes the implementation of each layer and the problems encountered while integrating them. Chapter 6 reports the verification results, including the automated suite, the interface checks and the defect log, and sets out the measurement protocol for recognition accuracy. Chapter 7 concludes, states the limitations honestly, and sets out the roadmap. The API reference, the bill of materials and the reference list are provided as appendices.

CHAPTER 2

Related Work

This chapter positions AutoPark against three bodies of work: how parking occupancy is sensed, how number plates are recognised, and how gate access and monitoring are implemented. The aim is not to catalogue the literature but to isolate the gap that this project addresses.

2.1 Occupancy sensing in smart parking

Lin, Rivano and Le Mouël classify smart parking systems by the sensing layer used to detect the presence of a vehicle [1]. The dominant approach is a sensor per bay: an ultrasonic, magnetometer, infrared or inductive-loop device mounted in or beside each space, reporting occupancy to a gateway. These systems are accurate because they measure the state of each bay directly, and commercial deployments are common in multi-storey facilities.

The economic structure of that approach is the problem. Every bay requires one device, one power drop, one mounting point and one more thing that can fail. For a facility with n bays, the sensing layer grows as $O(n)$ in capital cost, installation time and maintenance. Retrofitting an existing unmonitored lot is invasive, because cabling must reach every space.

A second family of designs uses vision from a vantage point: a camera overlooking a row or a level, with spaces delineated in the image and a classifier deciding occupancy per region. This amortises hardware over many bays, but each camera must see the spaces it serves, so coverage still scales with the geometry of the lot, and occlusion by large vehicles degrades accuracy. Camera-based occupancy systems also carry privacy and storage costs because they record the interior of the facility continuously.

Gate-based recognition, the approach taken here, sits between these families. The camera observes only the entrance and the exit, so two devices serve an arbitrary number of bays. Occupancy is not measured at all; it is inferred from the sequence of entries and exits. This is the same principle used by ticket-based and barrier-controlled commercial car parks, where the ticket count, rather than a sensor, is the occupancy record. The contribution of this project is to replace the ticket with an automatically recognised plate, which makes the record auditable without a human at the gate.

2.2 Automatic number plate recognition

Automatic number plate recognition (ANPR, also ALPR) has a long literature; Du et al. survey the classical pipeline of plate localisation, character segmentation and character recognition [3]. Classical systems rely on edge statistics, morphology and colour cues to find the plate, then on template matching or a small classifier per character. They work well under controlled illumination and fixed camera geometry, and degrade quickly when the plate is small, skewed or unevenly lit.

Two developments changed the practical design. First, single-stage object detectors such as YOLO reframed localisation as a regression problem solved in one network pass, giving

real-time detection at modest cost [4]. Modern implementations of the family, including YOLO11, are trained on plate-specific datasets and are accurate enough to replace the hand-crafted localisation stage entirely [5]. Second, deep-learning OCR engines removed the need for explicit character segmentation. EasyOCR detects text regions and decodes them with a recognition network, returning a confidence per token, which makes it suitable for unconstrained scene text such as a plate photographed by a low-cost camera [6].

A recognition pipeline assembled from these parts is accurate but not certain. That distinction matters for access control: a character error rate of a few per cent is acceptable for a toll audit, but a single wrong admit corrupts the occupancy state of a lot. The literature on ANPR evaluation therefore reports precision and recall per condition rather than a single accuracy figure, because performance is strongly condition-dependent [3].

The design consequence adopted in this project is to treat recognition as evidence rather than as an answer. Multiple preprocessing variants of the same crop are read independently, the readings are scored for confidence and agreement, several frames are accumulated, and the resulting candidates are compared against known registrations using a string-distance metric [8] rather than a single exact comparison. When the evidence does not resolve to one registration, the system refuses.

2.3 Low-cost edge vision and actuation

The ESP32 family provides a microcontroller with integrated Wi-Fi at a price point that makes a camera node disposable [9]. The ESP32-CAM variant adds an OV2640 sensor and PSRAM, enough to capture and upload a VGA JPEG without a companion processor. This class of device is widely used in IoT prototypes because it removes the need for a single-board computer at the edge: capture, transport and actuation can all run on one microcontroller.

Actuation at the gate is normally a barrier arm driven by a DC motor, a solenoid or a hobby servo. A servo is attractive for a prototype because it is position-controlled: the controller commands an angle rather than a direction, and no limit switches are needed to stop the arm. The SG90 class of micro-servo is cheap and adequate for a lightweight arm, at the cost of torque and of having no position feedback to the controller [10].

The architectural lesson taken from this literature is that the edge device should be a sensor and an actuator, not a decision maker. Every published deployment that puts authorisation logic on the camera node struggles to keep state consistent across nodes. AutoPark therefore keeps all decisions on the server and reduces the firmware to capture, upload, and move the arm when told.

2.4 Real-time monitoring interfaces

Operator dashboards need to reflect state changes within a second or two. The classical solution is client polling of a REST endpoint [15], which is simple but generates redundant requests and introduces a polling interval of latency. Server-sent events and WebSockets invert the direction: the server pushes each transition to connected clients. For a small number of screens and infrequent events, a single WebSocket channel is both simpler to operate than a message broker and more responsive than polling.

The relevant design detail, often omitted in student projects, is recovery. A WebSocket stream does not replay missed messages, so a client that reconnects after a network drop must re-read a snapshot before resuming the stream. This project implements that snapshot-then-stream pattern.

2.5 Access control, identity and audit

The web tier of a parking product is a conventional authenticated application. JSON Web Tokens [14] with a server-side signature provide stateless session identity, and password storage follows the OWASP recommendation of a slow, salted hash [16]. The framework choices — FastAPI for a typed HTTP API [11], SQLAlchemy for the persistence layer [13], and Next.js for the client [12] — are unremarkable; they are selected because they minimise the code needed for the parts of the system that are not the contribution.

The audit requirement is more interesting. A gate system must be able to answer, after the fact, why a particular vehicle was admitted or refused. That requires recording the evidence behind each decision, not only its outcome. This is the reason the schema contains a detection-event table separate from the session table, and the reason the recognition pipeline exposes per-stage diagnostics.

2.6 Research gap

Three observations follow from the survey.

First, occupancy sensing is the cost driver in smart parking, and virtually all published designs instrument each bay or each zone. Gate-only occupancy inference is used in practice by ticket-based operators but is rarely treated as an academic design, and its failure modes are rarely analysed.

Second, ANPR accuracy is usually reported as a single figure, while the property that matters for access control is the decision error rate — false admits and false rejects — under a refusal policy. A system that refuses when uncertain has a different error profile from one that always returns its best guess.

Third, the integration of recognition, authorisation, actuation, billing and real-time monitoring into one auditable pipeline is where most prototypes stop short.

AutoPark addresses these three points together: a two-camera architecture whose sensing cost is constant in the number of bays, a refusal-by-default authorisation policy with OCR-aware matching, and an end-to-end implementation whose decisions are recorded with evidence.

2.7 Comparison with related approaches

Table 2.1 — Comparison with related approaches

Approach	Occupancy accuracy	Hardware per bay	Retrofit effort	Cost scaling	Audit trail
Manual attendant	high	none	none	labour	paper
Per-bay ultrasonic/IoT	high	sensor + power + mount	high	linear	digital
Overhead camera CV (lot-wide)	medium–high	1 camera per zone	medium	stepwise	digital
AutoPark (gate ANPR)	derived	**none**	**gate only**	**constant**	**digital**

The honest positioning: AutoPark is not more accurate than per-bay sensing. It is **cheaper to scale and less invasive to retrofit**, at the cost of dependence on plate reads.

CHAPTER 3

Requirements Analysis

This chapter turns the problem statement into testable requirements, then examines whether the requirements can be met with the available hardware and skills.

3.1 Actors

Table 3.1 — Actors

Actor	Interaction with the system
Driver	Registers, pre-books a bay or enters as a walk-in, arrives at the entry gate, collects a receipt at exit
Operator / security staff	Monitors the lot, inspects recognition diagnostics, releases a stuck bay, resets the facility
Entry camera node	Captures frames, uploads them, opens the entry barrier on command
Exit camera node	Captures frames, uploads them, opens the exit barrier on command
Facility owner	Consumes occupancy and revenue records; not a direct user of the interface

3.2 Functional requirements

Each requirement is stated so that it can be verified by a test or by inspection. The identifier in the first column is used in the traceability matrix in Chapter 7.

Table 3.2 — Functional requirements

ID	Requirement	Acceptance criterion
FR-01	A driver shall be able to register and sign in with a username and password	A valid registration returns a token; invalid credentials are rejected with HTTP 401
FR-02	A signed-in driver shall be able to pre-book a bay, optionally choosing a specific bay	The response names the requested bay; the bay becomes ALLOCATED immediately
FR-03	The system shall refuse a booking when the requested bay is held by another vehicle	HTTP 400 with the bay identifier in the message
FR-04	The system shall refuse a second active booking for the same user or the same plate	HTTP 400 naming the existing bay
FR-05	The system shall capture a frame at the entry gate at a fixed interval	Firmware posts a multipart JPEG every 5 s
FR-06	The system shall locate and read a licence plate from the captured frame	Pipeline returns a candidate list with confidences, or a failure stage
FR-07	The system shall admit pre-booked vehicles to their held bay and dynamically allocate the first available bay to walk-in arrivals	Booked vehicle assigned held bay; walk-in assigned first free bay and admitted; full lot returns `parking_full`
FR-08	The system shall allocate the bay chosen at booking time when it is still free	Entry response slot equals the booked slot
FR-09	The system shall not double-allocate on repeated frames from the same vehicle	Repeat entry returns `already_allocated` without creating a second session
FR-10	The system shall open the barrier only on an explicit authorisation from the server	Firmware opens only when `open_barrier` is true
FR-11	The system shall derive bay occupancy from entry and exit events	Bay status changes only through a session being created or closed
FR-12	The system shall match a vehicle at exit against active sessions and release only that bay	Exit returns the matched slot; other bays are unchanged
FR-13	The system shall compute an itemised charge from the parking duration	60 min bills ₹30; 61 min bills ₹50; 121 min bills ₹70
FR-14	The system shall present live availability to the driver and the operator	A state change is visible on all open clients without a page refresh
FR-15	The system shall record every detection outcome with supporting evidence	A diagnostics record exists for each processed frame
FR-16	The system shall allow an operator to release a bay and reset the facility	Manual release frees one bay; reset frees all bays and preserves history
FR-17	The system shall support demonstration without physical cameras	Simulator endpoints produce the same allocation, billing and event path

3.3 Non-functional requirements

Table 3.3 — Non-functional requirements

ID	Requirement	Target	How it is addressed
NFR-01	Correctness of occupancy	No bay shown occupied without an active session	Occupancy derived from the session table only; single-writer allocation under a lock
NFR-02	Safety of authorisation	Zero silent false admits on ambiguous evidence	Four independent refusal gates; ambiguity is a refusal by construction
NFR-03	Responsiveness	Occupancy change visible in under 2 s	WebSocket broadcast on every state transition
NFR-04	Availability	Backend restart does not lose lot state	All state persisted in SQLite; cameras retry every 5 s
NFR-05	Fail-safe behaviour	Barrier stays closed when the backend is unreachable	Firmware opens only on an explicit server boolean
NFR-06	Usability	A driver can find a bay and complete a booking without instruction	Single primary action per card; bay pre-selection carried through the booking form
NFR-07	Auditability	Every decision can be reconstructed after the fact	Detection-event table with plate, confidence, stage and evidence reference
NFR-08	Portability	Runs on commodity hardware on a facility LAN	Python and Node services; no cloud dependency; SQLite file database
NFR-09	Maintainability	Recognition thresholds and slot count configurable without code edits	Central `settings` object and a single slot list
NFR-10	Cost	Sensing hardware independent of the number of bays	Two camera nodes regardless of capacity

3.4 Constraints

Table 3.4 — Constraints

Constraint	Consequence for the design
Two hobby-grade camera modules only	Recognition must tolerate compression, motion blur and limited resolution; a refusal path is mandatory
Single laptop hosts both services	No horizontal scaling; in-process locking is sufficient but not more
Facility LAN, no public address	No TLS, no external payment callback; documented as a limitation
SG90 torque and 5 V supply	Lightweight arm only; no position feedback available
Academic timeline	Payment gateway integration deferred to future work; walk-in allocation dynamically enabled via `ALLOW_WALKIN_ENTRY` policy
Indian plate formats	Matching must tolerate state prefixes, two-line plates and visually confusable characters

3.5 Feasibility

Technical feasibility. Every component used is available as an open-source library or a commodity module: FastAPI and SQLAlchemy for the service layer, OpenCV and a YOLO plate model for localisation, EasyOCR for recognition, and the Arduino core for the ESP32-CAM. The integration risk is concentrated in recognition quality on low-cost hardware, which is why the design includes an explicit refusal path and a diagnostics interface rather than assuming success.

Economic feasibility. The bill of materials is dominated by two ESP32-CAM modules and two servos; jumper wire, a programmer and mounting material complete the rig. A conventional design for the same six bays would require six occupancy sensors, six mounting points and, in most cases, six cable runs. The gate-based design therefore reduces both the part count and the installation effort, and its marginal sensing cost for an additional bay is zero.

Operational feasibility. The system runs unattended. The only routine human actions are the exceptions that any gate system has: releasing a bay when a plate could not be read at exit, and resetting the facility between demonstration runs. Both are exposed in the operator console and both are recorded.

3.6 Scope boundary

The requirements above deliberately exclude two capabilities that a commercial product would need: online payment processing, and multi-lot management. Each is excluded because it introduces payment compliance or multi-tenant infrastructure work that does not test the central hypothesis of the project — that occupancy can be derived from two gate cameras instead of a sensor per bay. Unreserved walk-in entry is fully integrated and supported alongside advance reservations.

CHAPTER 4

System Design

4.1 Architectural style

AutoPark is an **edge-gateway, event-driven architecture** with a thin browser client.

- **Edge tier** — two ESP32-CAM nodes. They do exactly two things: capture a JPEG every five seconds and actuate a servo when told to. They hold no state about the lot.
- **Application tier** — a FastAPI service that owns all decisions: identity, plate matching, bay allocation, billing and audit. It is the single source of truth.
- **Presentation tier** — a Next.js application that renders state and reacts to WebSocket events. It never computes occupancy.
- **Data tier** — SQLite through SQLAlchemy, with five tables.

The important architectural consequence: **cameras are sensors, not controllers.** If a camera is replaced, the lot state is unaffected. If the backend is down, the barriers fail closed.

4.2 Context diagram

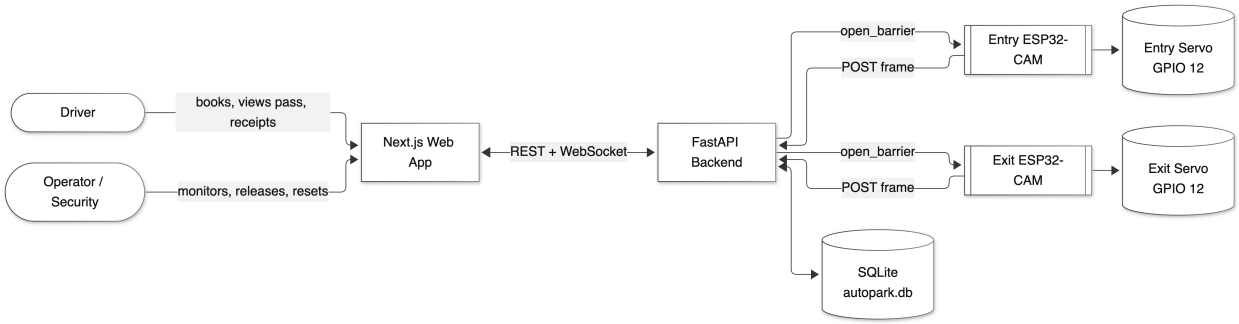


Figure 4.1 — Context diagram

4.3 Container / component view

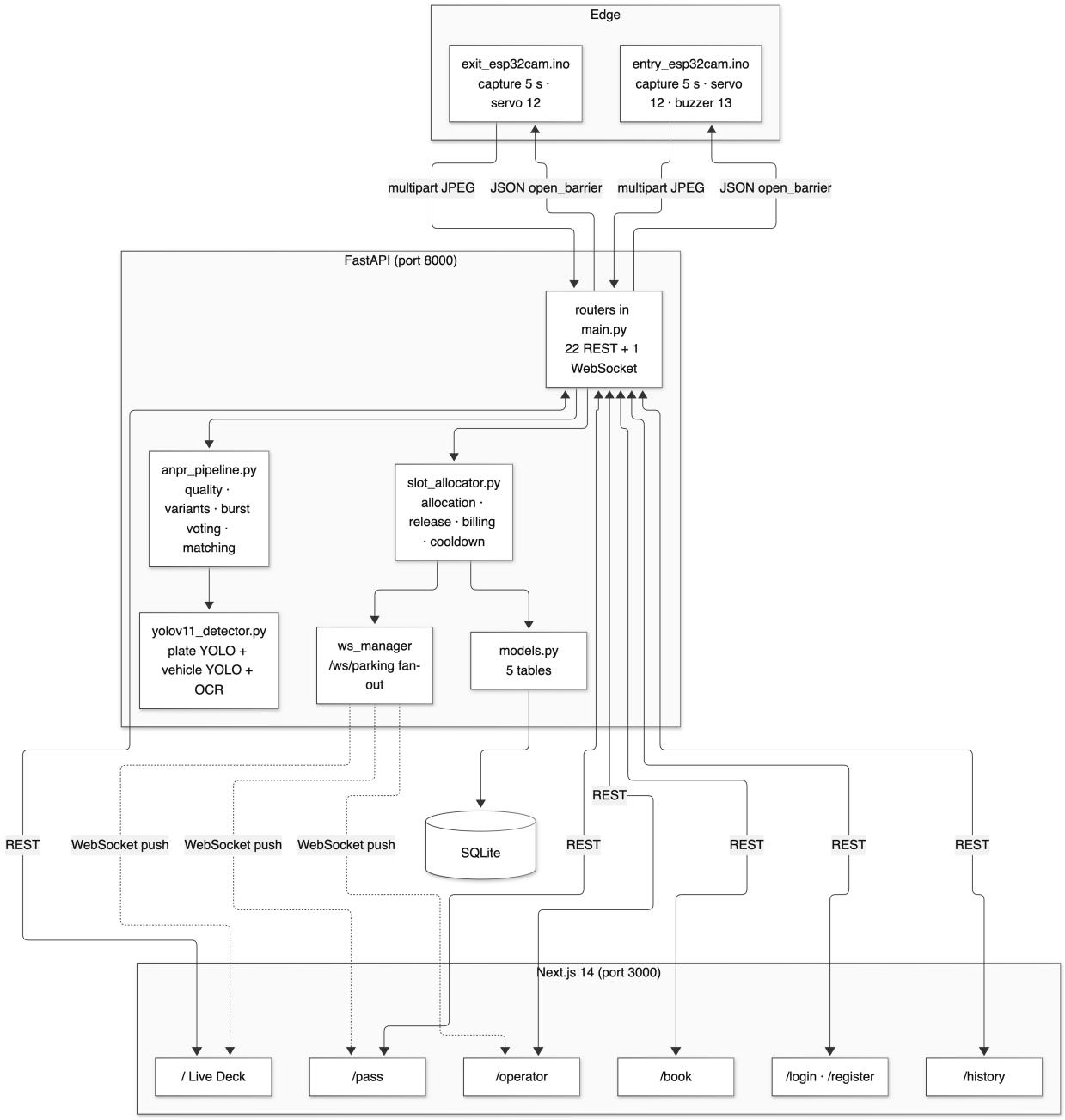


Figure 4.2 — Container / component view

4.4 End-to-end data flow

4.4.1 Booking (before arrival)

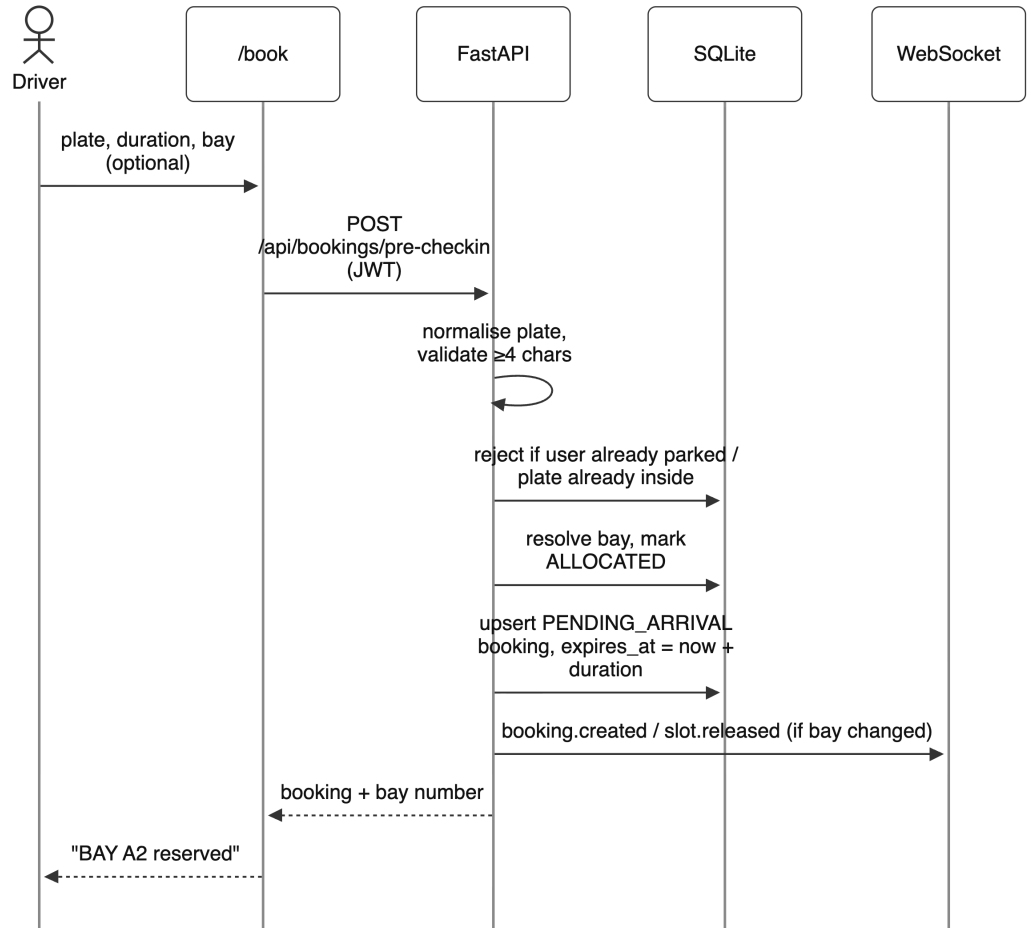


Figure 4.3 — Booking (before arrival)

Key rule: **booking consumes the bay immediately.** A reserved bay is not offered to anyone else, even though the car has not arrived.

4.4.2 Entry (authorised)

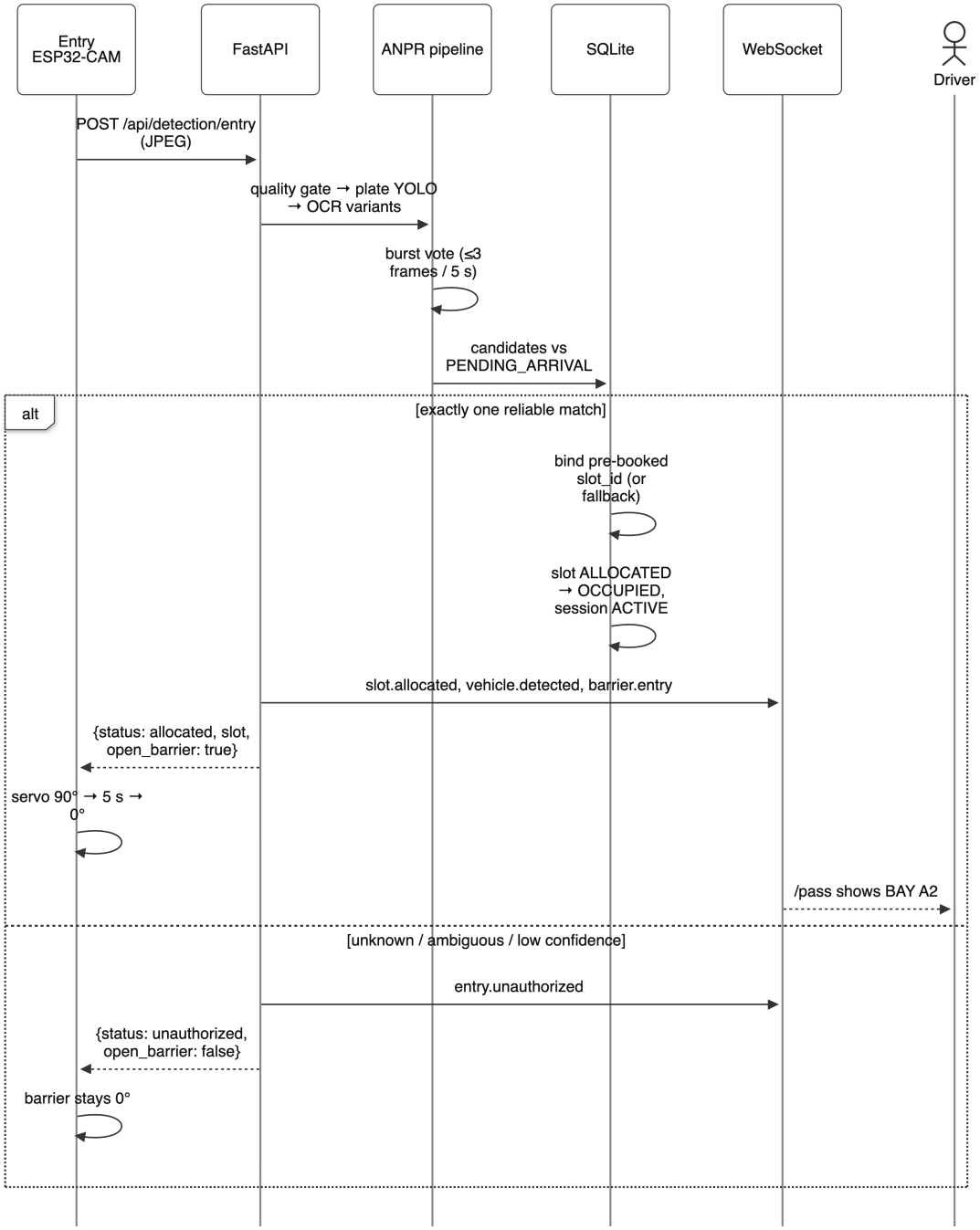


Figure 4.4 — Entry (authorised)

4.4.3 Exit

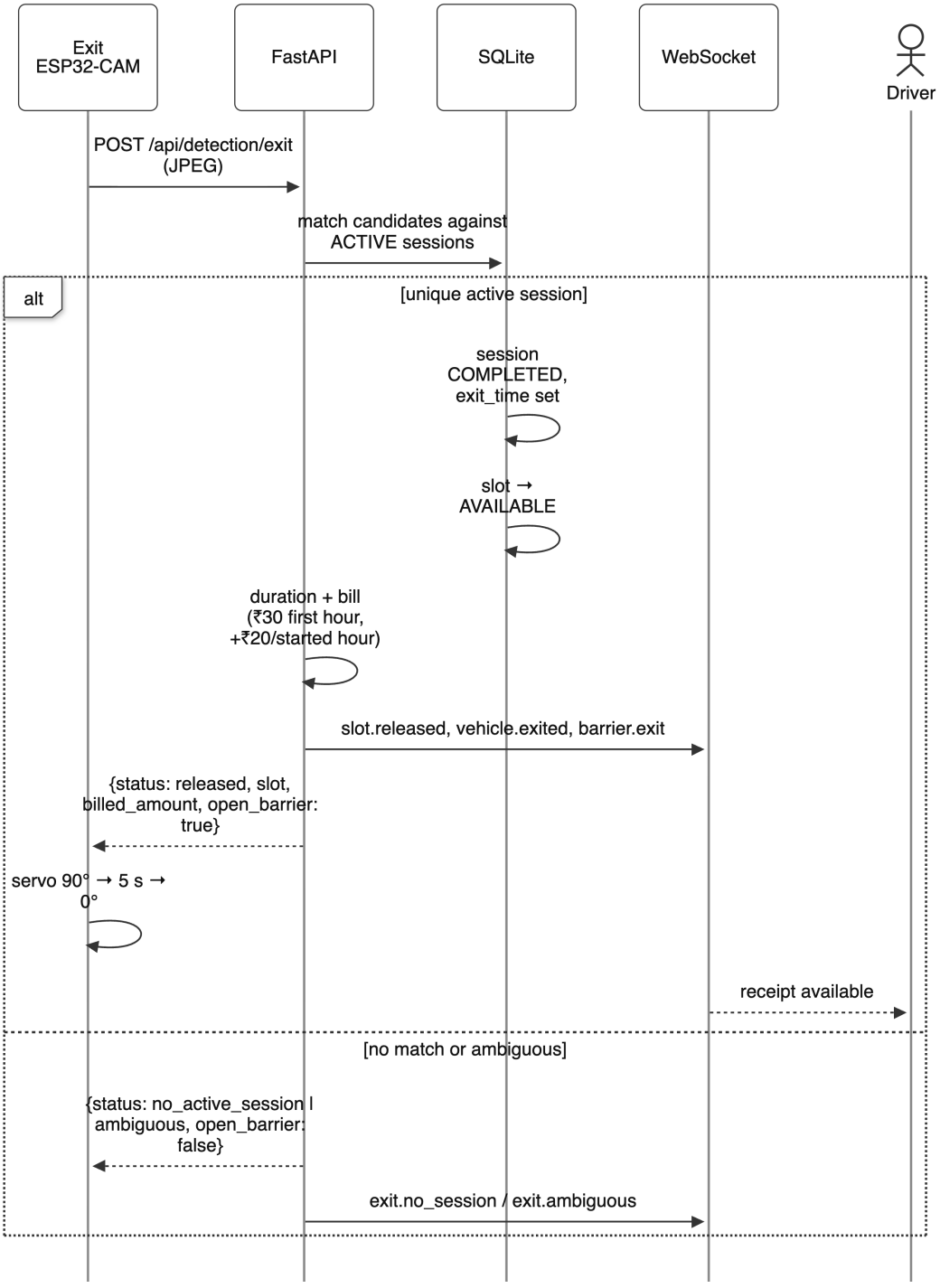


Figure 4.5 — Exit

4.4.4 Realtime fan-out

Every state change is broadcast on `/ws/parking`. Clients (Live Deck, Active Pass, Operator Console) subscribe once and patch local state. There is no polling loop for occupancy.

Event types actually emitted by the backend:

Table 4.1 — Realtime fan-out

Event	Meaning
`slot.allocated`	A bay changed to ALLOCATED/OCCUPIED
`slot.released`	A bay became AVAILABLE
`booking.created`	Pre-registration created
`booking.cancelled`	Pre-registration cancelled
`vehicle.detected`	A frame produced a detection result
`vehicle.exited`	Exit matched an active session
`entry.unauthorized`	Entry refused
`exit.no_session`	Exit plate matched nothing
`exit.ambiguous`	Exit matched more than one session
`parking.full`	No bay available
`barrier.entry` / `barrier.exit`	Barrier action for the UI indicator
`buzzer.triggered`	Optional buzzer fired
`demo.reset`	Facility reset

4.5 State machines

4.5.1 Parking bay

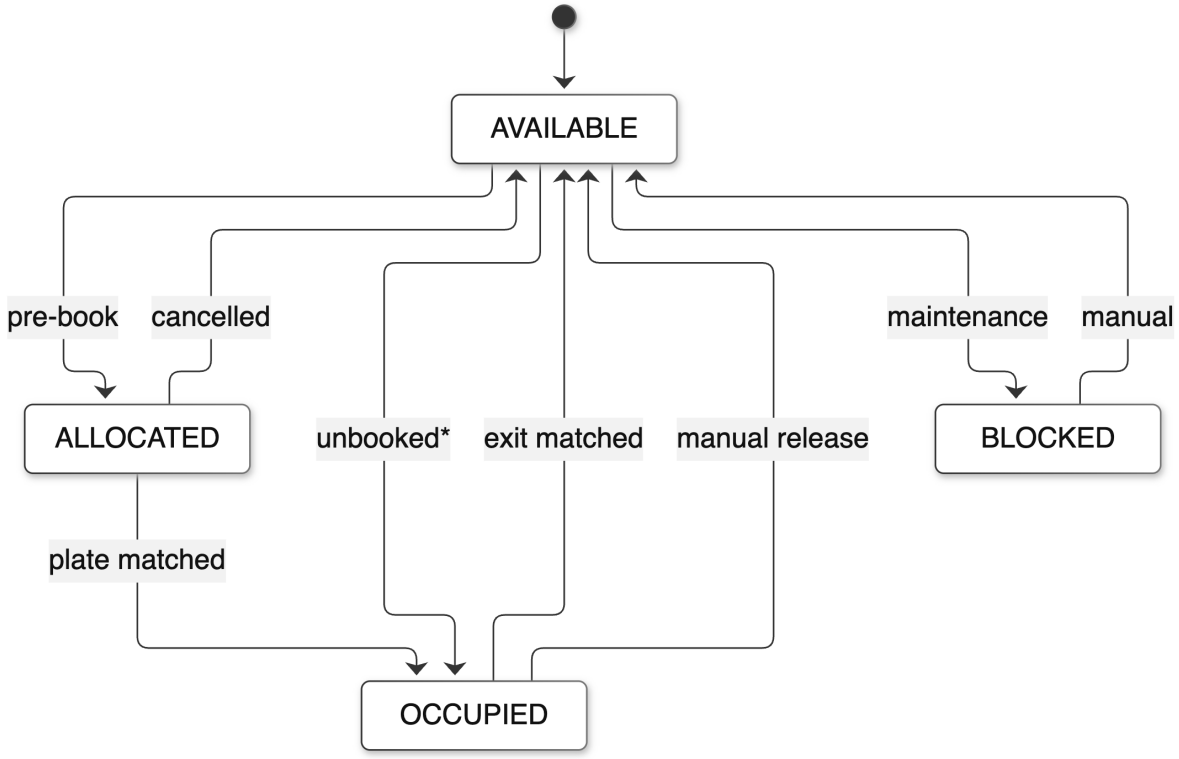


Figure 4.6 — Parking bay

4.5.2 Booking

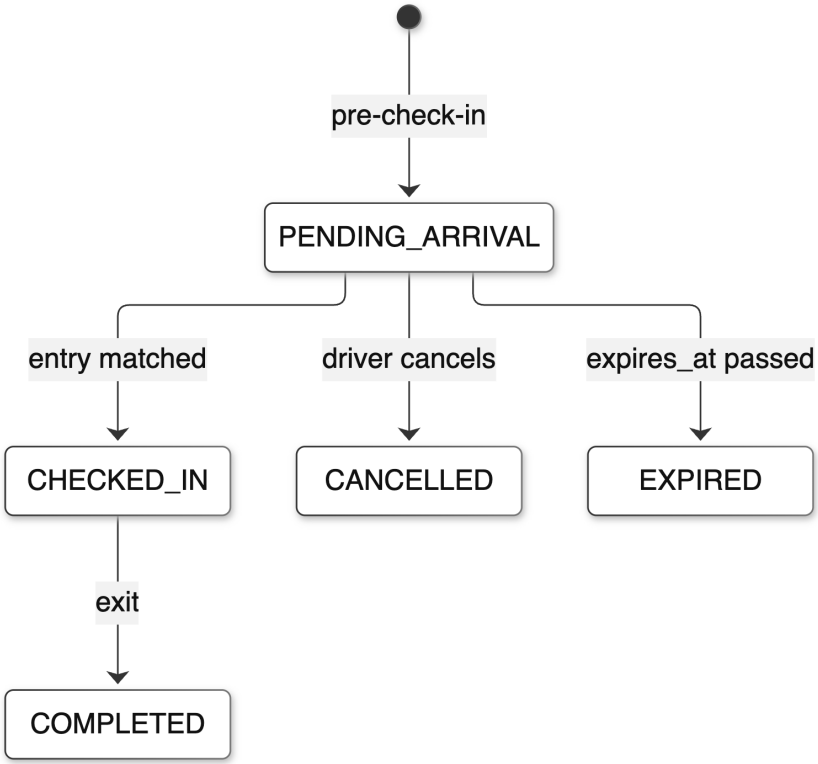


Figure 4.7 — Booking

4.5.3 Session

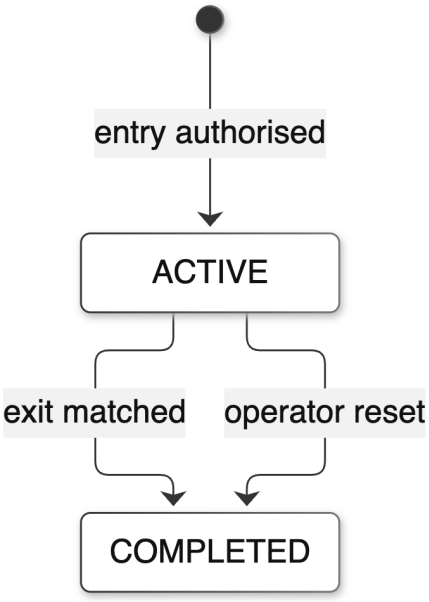


Figure 4.8 — Session

4.6 Deployment topology

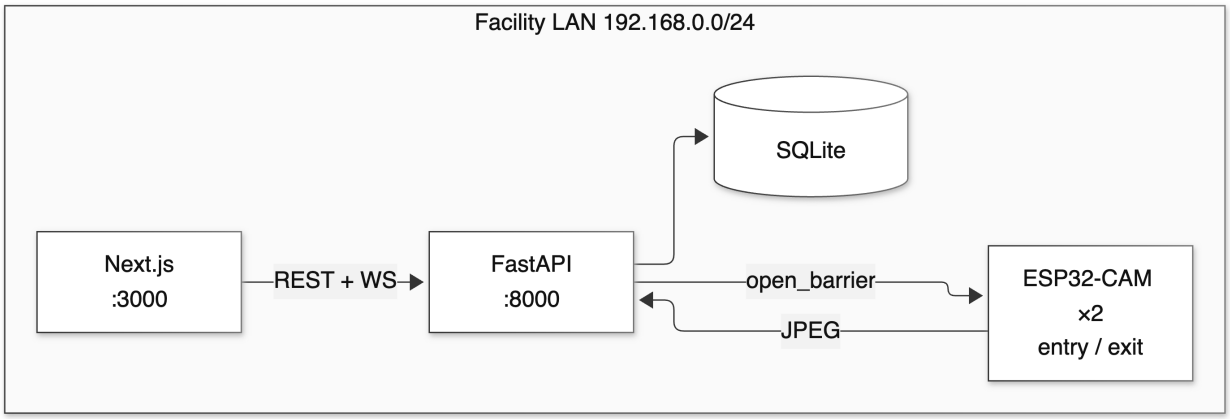


Figure 4.9 — Deployment topology

- No cloud dependency, no external database, no TLS terminator.
- Cameras are configured with the laptop's LAN address in firmware (`SERVER_HOST`, `SERVER_PORT`, `SERVER_PATH`).
- Both services are started detached with `scripts/host.sh`; logs in `.run/`.
- This is a **LAN deployment**, which is the realistic topology for a gate system. It is not an internet-hosted service.

4.7 Database schema

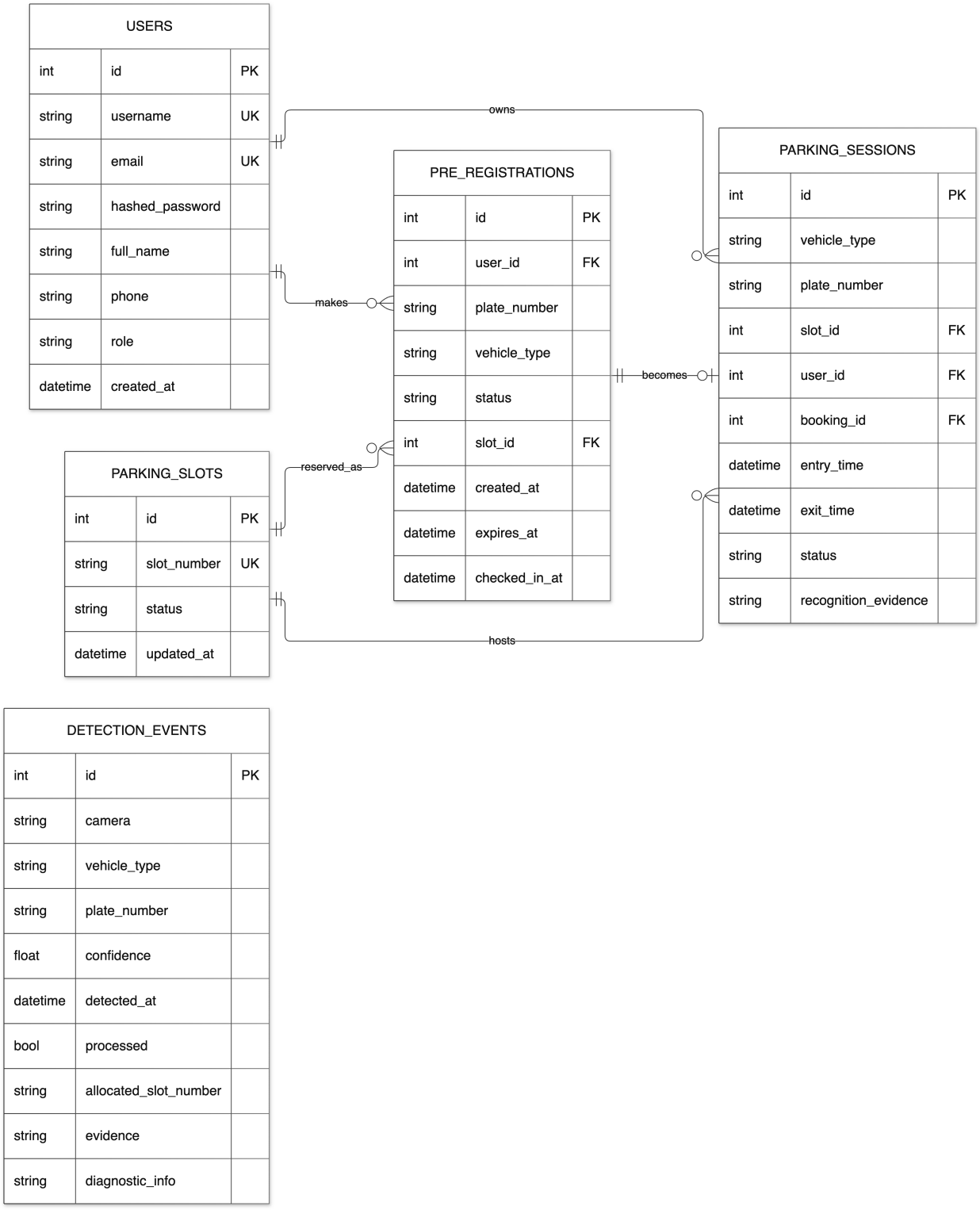


Figure 4.10 — Database schema

4.8 API reference

Base URL: `http://localhost:8000` · interactive docs: `/docs`

4.8.1 System

Table 4.2 — System

Method	Path	Auth	Purpose
GET	`/`	—	Service banner
GET	`/health`	—	Health, model status, slot and session counts
GET	`/api/health`	—	Alias of `/health`

4.8.2 Detection (edge cameras)

Table 4.3 — Detection (edge cameras)

Method	Path	Auth	Body	Purpose
POST	`/api/detection/entry`	—	multipart JPEG or form fields	Run ANPR, authorise, allocate, return barrier command
POST	`/api/detection/exit`	—	multipart JPEG or form fields	Run ANPR, match session, release, bill

Form fallback fields: `vehicle_type`, `plate_number`, `confidence`, `sync`. The firmware always uploads a JPEG.

4.8.3 Authentication

Table 4.4 — Authentication

Method	Path	Auth	Purpose
POST	`/api/auth/register`	—	Create account, return JWT
POST	`/api/auth/login`	—	Verify credentials, return JWT
GET	`/api/auth/me`	Bearer	Current user profile

4.8.4 Bookings

Table 4.5 — Bookings

Method	Path	Auth	Purpose
POST	`/api/bookings/pre-checkin`	Bearer	Create or update a booking, bind a bay
GET	`/api/bookings/my-status`	Bearer	Active booking + active session
GET	`/api/bookings/my-history`	Bearer	Last 20 sessions → `{ "history": [...] }`
GET	`/api/bookings/pending`	—	All non-expired pending bookings (operator)
DELETE	`/api/bookings/{id}`	Bearer	Cancel own booking, release its bay

4.8.5 Parking state and control

Table 4.6 — Parking state and control

Method	Path	Auth	Purpose
GET	`/api/parking/layout`	—	Slots with status and active session
GET	`/api/parking/slots`	—	Raw slot list
GET	`/api/parking/summary`	—	Counts: total, available, allocated, occupied, blocked
POST	`/api/parking/reset`	—	Reset facility: complete sessions, free bays, preserve history
POST	`/api/parking/release/{slot_number}`	—	Manual release of a stuck bay
POST	`/api/parking/simulate/entry`	—	HIL: authorised or unauthorised entry
POST	`/api/parking/simulate/exit`	—	HIL: exit for the active session

4.8.6 Diagnostics

Table 4.7 — Diagnostics

Method	Path	Purpose
GET	`/api/anpr/diagnostics?limit=N`	Last N detection diagnostics (quality, candidates, timings, failure stage)
GET	`/api/anpr/debug-image/{name}`	Retrieve a stored debug capture

4.8.7 WebSocket

Table 4.8 — WebSocket

Path	Direction	Purpose
`/ws/parking`	server → client	Push all state-change events

4.9 Pipeline overview

The pipeline has two halves. The first turns a camera frame into candidate plate readings. The second decides whether those readings are strong enough to open the barrier. Both halves run inside a single request, so a frame is never left half-processed.

4.9.1 Capture and reading (stages 1–4)

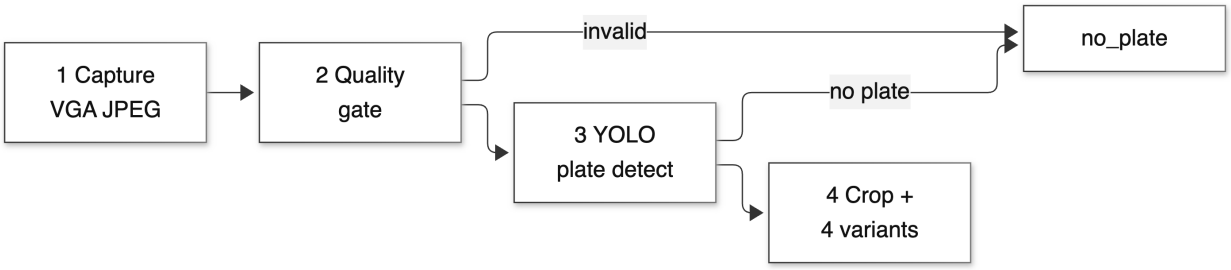


Figure 4.11 — Capture and reading (stages 1–4)

4.9.2 Decision path (stages 5–8)

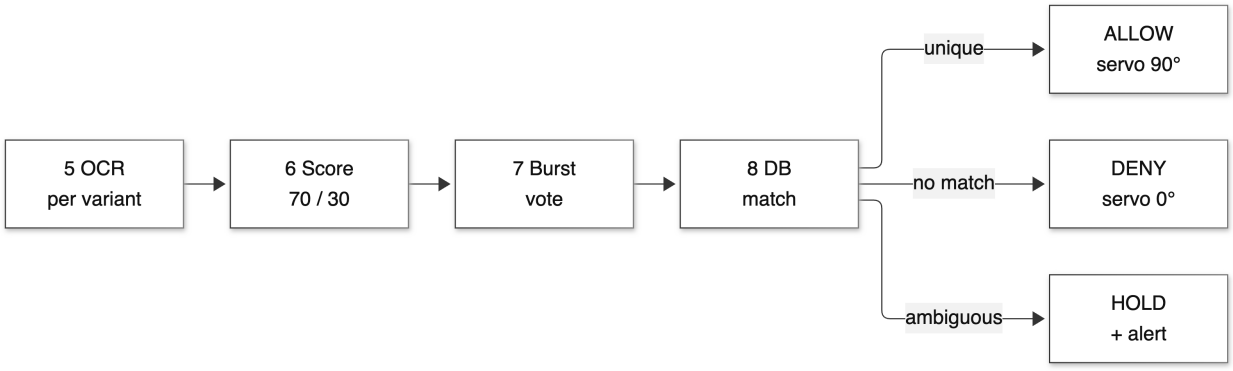


Figure 4.12 — Decision path (stages 5–8)

4.9.3 Where each stage lives

Stages live in:

Table 4.9 — Where each stage lives

Stage	File	Function
Quality gate	`anpr_pipeline.py`	`assess_image_quality()`
Preprocessing variants	`anpr_pipeline.py`	`generate_plate_variants()`
OCR + candidate scoring	`anpr_pipeline.py`	`process_frame()` / `extract_candidates()`
Burst accumulation	`anpr_pipeline.py`	`FrameBurstAccumulator`
Entry matching	`anpr_pipeline.py`	`match_entry_pre_registration()`
Exit matching	`anpr_pipeline.py`	`match_exit_active_session()`
Allocation policy	`slot_allocator.py`	`allocate_entry_slot()`
Release + billing	`slot_allocator.py`	`release_exit_slot()`

4.10 Stage 7 — Database matching

4.10.1 Similarity scoring

`evaluate_plate_match(candidate, target)` returns:

Table 4.10 — Similarity scoring

Condition	Score	Example
Exact match	**1.00**	`KA05KK0403` = `KA05KK0403`
Canonical match	**0.92**	`KA05KK04O3` → `KA05KK0403`
Substring / prefix (≥6 chars)	**0.88**	`INDKA01AB1234` contains `KA01AB1234`
Levenshtein distance 1, length ≥6	**0.85**	one substitution
Levenshtein distance 1, length 4–5	**0.75**	one substitution on a short plate
Levenshtein distance 2, length ≥7	**0.76**	two edits on a long plate
Otherwise	**0.00**	—

4.10.2 Canonicalisation map

OCR confuses characters that look alike. Before comparison:

```
I, L, J → 1
O, Q, D → 0
Z → 2
S → 5
B → 8
G → 6
```

So `KA05KK04O3` and `KA05KK0403` canonicalise to the same string. This is why a single `O`/`0` error does not reject a legitimate driver.

4.10.3 Confidence weighting

The raw similarity is discounted by the OCR's own confidence:

```
adjusted_score = similarity × (0.80 + 0.20 × candidate_confidence)
```

A perfect string match read at 0.50 confidence scores `1.00 × 0.90 = 0.90`; the same match at 0.95 scores `0.99`. The floor of 0.80 prevents a confident wrong read from being discounted into a match, and the ceiling prevents a marginal read from outranking a strong one.

4.10.4 Acceptance and ambiguity

Table 4.11 — Acceptance and ambiguity

Rule	Value
Minimum adjusted score to be considered	**0.72**
Ambiguity condition	top two scores within **0.10**
Ambiguous outcome	**refuse**, raise operator alert
No candidate above threshold	`unauthorized` (entry) / `no_active_session` (exit)

The ambiguity rule is the most important safety property in the project. If a reading could plausibly belong to two bookings, the system does **not** pick the higher one — it stops. A false admit creates a wrong bay, a wrong bill and a corrupted occupancy map; a false reject costs one retry.

Worked example — ambiguity

```
Bookings: KA05KK0403, KA05KK0404
Reading : KA05KK0405  (one edit from each)
Scores  : 0.85, 0.85  → gap 0.00 < 0.10 → AMBIGUOUS → refuse
```

Worked example — canonical match

```
Booking: KA05KK0403
Reading: KA05KK04O3   (O instead of 0)
Canonical both → KA05KK0403 → similarity 0.92
candidate confidence 0.91 → adjusted 0.92 × 0.982 = 0.90 ≥ 0.72 → ALLOW
```

4.11 Stage 8 — Policy layer (allocation)

Matching produces a candidate booking; `allocate_entry_slot()` applies policy **in this order**:

1. **Confidence gate** — `< 0.75` → `ignored`.
2. **Plate validity** — no candidate with ≥4 alphanumerics → `no_plate`.
3. **Already inside?** — plate matches an ACTIVE session → `already_allocated` (idempotent duplicate-frame handling).

4. **Cooldown** — same canonical plate within 10 s → `already_allocated`.
5. **Booking match or walk-in policy** — unique non-ambiguous booking matches held bay; if no booking exists and `ALLOW_WALKIN_ENTRY` is active, the first available bay is dynamically allocated. If lot is full, returns `parking_full`.
6. **Bay binding** — use the booking's `slot_id` if it is still AVAILABLE/ALLOCATED; otherwise assign the lowest-numbered free bay.
7. **Commit** — bay → OCCUPIED, session ACTIVE, booking CHECKED_IN (if booked), `DetectionEvent` written with evidence.
8. **Actuate** — response includes `open_barrier: true`, `barrier_action: OPEN`.

Every branch returns a machine-readable status, so the UI and the firmware never have to parse prose.

4.12 Failure taxonomy

Table 4.12 — Failure taxonomy

Status	Cause	Barrier	Operator sees
`no_plate`	no plate box, unreadable, fewer than 4 alphanumerics (entry), bad quality	closed	"no plate" + stage
`ignored`	confidence < 0.75	closed	low-confidence event
`unauthorized`	plate matches no booking when walk-in is disabled	closed	driver + plate + alert
`ambiguous`	two bookings within 0.10	closed	both candidate plates
`already_allocated`	duplicate frame / cooldown / already inside	closed	existing bay
`parking_full`	no free bay	closed	full-lot alert
`released`	exit matched	open	bay, duration, bill
`no_active_session`	exit plate matches no ACTIVE session	closed	plate + alert
`exit ambiguous`	two sessions within 0.10	closed	both plates

4.13 Pin map (both nodes)

Table 4.13 — Pin map (both nodes)

Signal	ESP32-CAM pin	Notes
Servo signal (orange/yellow)	**GPIO 12**	PWM control; 0° closed, 90° open
Servo VCC (red)	**5 V rail**	*not* 3.3 V — brown-outs and jitter otherwise
Servo GND (brown/black)	**GND**	must share ground with ESP32-CAM
Buzzer + (optional)	**GPIO 13**	active buzzer; `-1` disables in firmware
Buzzer −	GND	—
Camera	internal AI-Thinker pin map	`Y2…Y9`, `XCLK`, `PCLK`, `VSYNC`, `HREF`, `SIOD`, `SIOC`, `PWDN`, `RESET`

GPIO 12 and 13 are free on the AI-Thinker camera pinout, which is why they were chosen.

4.13.1 Power topology

```
5 V 2 A adapter ─┬─── ESP32-CAM 5V
                └─── Servo VCC
GND ────────────┬─── ESP32-CAM GND
                └─── Servo GND
```

- Common ground is mandatory; without it the PWM signal has no reference.
- A 470 µF–1000 µF electrolytic capacitor across the 5 V rail near the servo reduces brown-out resets when the arm starts moving.
- Do **not** power the servo from the ESP32-CAM 3.3 V pin.

4.14 Wiring table — Entry node

Table 4.14 — Wiring table — Entry node

From	To	Purpose
ESP32-CAM 5V	5 V rail	board power
ESP32-CAM GND	common GND	reference
GPIO 12	servo signal	barrier PWM
5 V rail	servo VCC	torque supply
common GND	servo GND	return
GPIO 13	buzzer +	confirmation chirp (optional)
GND	buzzer −	return

4.15 Wiring table — Exit node

Identical, without the buzzer:

Table 4.15 — Wiring table — Exit node

From	To	Purpose
ESP32-CAM 5V	5 V rail	board power
ESP32-CAM GND	common GND	reference
GPIO 12	servo signal	barrier PWM
5 V rail	servo VCC	torque supply
common GND	servo GND	return

4.16 Barrier control contract

Backend response fields:

```
{
  "status": "allocated",
  "slot": "A2",
  "open_barrier": true,
  "barrier_action": "OPEN",
  "trigger_buzzer": true
}
```

Firmware decision (entry):

```
bool open_barrier = doc["open_barrier"] | false;
if (open_barrier || trigger_buzzer) {
```

```
    openEntryBoomBarrier();      // 90° → 5000 ms → 0°
} else {
    // gate stays at 0°
}
```

Exit is equivalent, keyed on `open_barrier || status == "released"`.

4.16.1 Truth table

Table 4.16 — Truth table

Backend outcome	open_barrier	Entry barrier	Exit barrier
allocated (valid booking)	true	**OPEN 90° / 5 s**	—
released (valid session)	true	—	**OPEN 90° / 5 s**
unauthorized	false	closed 0°	—
no_plate	false	closed 0°	—
ambiguous	false	closed 0°	closed 0°
no_active_session	false	—	closed 0°
parking_full	false	closed 0°	—
already_allocated	false	closed 0° (already inside)	—

The barrier is **fail-closed**: absence of an explicit `true` leaves the arm down.

4.17 Why two cameras instead of per-bay sensors

Table 4.17 — Why two cameras instead of per-bay sensors

Dimension	Per-bay sensors	AutoPark (gate cameras)
Sensing devices	one per bay (6 here, n in general)	**2, constant**
Power/cabling per bay	required	none
Mounting per bay	required	none
Occupancy accuracy	direct measurement	derived; relies on plate reads
Failure blast radius	one bay	one gate (half the flow)
Unreadable plate	irrelevant	bay can appear stuck until override
Cost growth with capacity	linear	constant
Retrofit into existing lot	invasive	gate-mounted only

The honest trade-off: direct sensing is more robust per bay. AutoPark exchanges per-bay robustness for constant hardware cost and a single integration point, and pays for it with a dependency on plate recognition quality. That dependency is managed with confidence thresholds, ambiguity refusal, cooldowns, evidence capture and manual release — not hidden.

4.18 Design decisions

Table 4.18 — Design decisions

ID	Decision	Alternatives considered	Rationale	Trade-off accepted
D1	Two gate cameras, no bay sensors	ultrasonic per bay; overhead vision	constant hardware cost; minimal retrofit	occupancy depends on plate reads
D2	Hybrid pre-booked & walk-in access	pre-booked only	accommodates advance reservations while serving spontaneous arrivals dynamically	walk-ins cannot take reserved bays
D3	Backend owns all state	camera-side decisions	single source of truth; auditable	backend is a single point of failure
D4	Refuse on ambiguity	best-score admit	a false admit corrupts occupancy and revenue	occasional false rejects
D5	SQLite	PostgreSQL	zero-ops, adequate for one lot	limited write concurrency
D6	JWT + bcrypt	session cookies	stateless API, simple ESP32/browser split	token revocation is manual
D7	WebSocket push	polling	instant UI, less load	connection lifecycle code
D8	Servo barrier on GPIO 12	buzzer only	physical access control, matches real gates	5 V current and timer contention care
D9	Deterministic billing in backend	client-side estimate	single authority; receipts cannot be edited	recomputation on every exit
D10	Simulation endpoints	mock UI only	exercises the real DB/allocator/billing path	simulator can create real sessions

4.19 Architectural invariants

These must remain true; violating one is a design regression.

1. **No bay is allocated without a plate decision.** The only exception is the explicit operator simulator.
2. **A bay is OCCUPIED only while an ACTIVE session references it.**
3. **Occupancy is never computed in the frontend.**
4. **The barrier opens only on an explicit backend boolean.**
5. **Every entry, exit and refusal writes a `DetectionEvent` row with evidence.**
6. **A booking always holds a concrete `slot_id`; releasing a booking releases the bay.**
7. **Ambiguity is a refusal, not a coin flip.**

4.20 Failure and degradation design

Table 4.19 — Failure and degradation design

Failure	System behaviour	Recovery
Backend unreachable	Cameras retry on next 5 s cycle; barrier stays closed (fail-safe)	restart backend
Camera offline	Last-seen status ages out after 30 s (`CAMERA_OFFLINE_TIMEOUT_SECONDS`)	power-cycle camera
Blurred / dark frame	Quality gate marks frame `blurry` / `poor_lighting`; no allocation	better lighting / refocus
Plate unreadable	`no_plate`; no allocation; event logged with evidence	manual release or retry
Plate matches two bookings	`ambiguous`; refused	operator verifies
Entry read succeeds, exit read fails	Bay remains OCCUPIED	operator manual release (`POST /api/parking/release/{slot}`)
Duplicate frames	10 s per-plate cooldown + active-session check	automatic
Power loss mid-session	Session stays ACTIVE in DB	reset or manual release
Servo stalls	No position feedback; UI label is inferred, not measured	visual check / re-seat horn

The barrier is deliberately **fail-closed**: the absence of an explicit `open_barrier: true` from the backend leaves the arm at 0°.

CHAPTER 5

Implementation

5.1 Stack at a glance

Table 5.1 — Stack at a glance

Layer	Technology	Version in repo	Role
Edge firmware	Arduino C++ (ESP32 core)	—	Capture + HTTP POST + servo control
Edge hardware	ESP32-CAM (AI-Thinker), SG90 servo, optional buzzer	—	Gate sensing and actuation
Plate detection	YOLOv11 (Ultralytics)	`ultralytics` (not pinned)	Locate plate region in frame
Plate reading	EasyOCR	(not listed in requirements)	Recognise characters
Image processing	OpenCV (headless)	`>=4.8.0`	Quality metrics, CLAHE, thresholding
Backend framework	FastAPI	`>=0.100.0`	REST + WebSocket API
ASGI server	Uvicorn	`>=0.22.0`	Serve the API
ORM	SQLAlchemy	`>=2.0.0`	Models and queries
Database	SQLite	stdlib	Persistence
Auth	PyJWT + bcrypt	`>=2.8.0`, `>=4.0.0`	Tokens and password hashing
Frontend framework	Next.js (App Router)	`^14.2.5`	Routing, rendering
UI runtime	React	`^18.3.1`	Components and state
Styling	Tailwind CSS	`^3.4.7`	Design system utilities
Icons	lucide-react	`^0.417.0`	Consistent line icons
Motion	Framer Motion	`^11.3.19`	Modal / card transitions
Language	TypeScript	`^5.5.4`	Type safety on the client

5.2 Version selection

- **Next.js 14 App Router** — the repo already uses server/client component boundaries; upgrading majors would churn every page for no functional gain.
- **React 18** — required by Next 14; concurrent rendering is not used explicitly.
- **SQLAlchemy 2.x** — the models use the 2.0 declarative style.
- **Pydantic 2.x** — FastAPI's response models and settings rely on it.
- **Tailwind 3.4** — custom colour tokens (`districtBg`, `districtCard`, …) are defined in `tailwind.config.js`.

5.3 Runtime topology and practical constraints

Table 5.2 — Runtime topology and practical constraints

Concern	Reality in this build
Backend interpreter	`/opt/anaconda3/bin/python3` — the default `python3` lacks SQLAlchemy
Backend start	`scripts/host.sh start` (detached), or `uvicorn app.main:app --port 8000`
Frontend start	`npm run dev` in `frontend/` (port 3000)
Database file	`backend/autopark.db` (root `autopark.db` is stale — no `users` table)
Camera → backend	plain HTTP on the LAN, no auth on `/api/detection/*`
Browser → backend	JWT bearer for account routes; occupancy routes are open
Public exposure	none — LAN only

> It is an environment issue, not an architectural one, and it is fixed by pinning the interpreter in `scripts/host.sh`.

5.4 Module map

Table 5.3 — Module map

Module	Responsibility
`main.py`	App setup, 22 REST routes, 1 WebSocket route, startup warm-up, WebSocket broadcasts
`config.py`	Settings: slot list, thresholds, cooldown, JWT, database URL
`models.py`	Five SQLAlchemy models
`database.py`	Engine, session factory, `get_db` dependency
`schemas.py`	Pydantic request/response contracts
`auth.py`	bcrypt hashing, JWT creation/decoding, `get_current_user`
`slot_allocator.py`	Allocation, release, billing, cooldown, manual release, reset
`anpr_pipeline.py`	Quality gate, variants, OCR candidates, burst voting, DB matching
`yolov11_detector.py`	Plate YOLO + vehicle YOLO + OCR reader lifecycle
`yolo_detector.py`	Fallback detector and EasyOCR reader accessor
`frame_processor.py`	Per-camera frame processing wrapper
`train_yolov11.py`	Offline training script (not part of runtime)
`ws_manager.py`	WebSocket connection registry and broadcast

5.5 Entry allocation algorithm

```
allocate_entry_slot(plate, confidence, candidates):
    if confidence < 0.75:                return "ignored"
    if no candidate with >= 4 chars:     return "no_plate"
    lock allocation_lock:
        if plate matches an ACTIVE session:  return "already_allocated"
        if same canonical plate within 10 s: return "already_allocated"
        match = unique match among PENDING_ARRIVAL bookings
        if match is ambiguous:           return "ambiguous"
        if match is none:
            if not ALLOW_WALKIN_ENTRY:   return "unauthorized"
            booking = None               # proceed with dynamic walk-in allocation
        bay = booking.slot_id if (booking and booking.slot_id)
              else lowest-numbered AVAILABLE bay
        if no bay:                       return "parking_full"
        bay.status = OCCUPIED
        if booking: booking.status = CHECKED_IN
        create session(ACTIVE, plate, bay, booking, evidence)
        log DetectionEvent
        broadcast slot.allocated (signals UI to display bay callout & highlight card)
        return { status: "allocated", slot: bay, open_barrier: true }
```

Notes:

- The **bay the driver booked wins** over the lowest-numbered bay. This was a real bug, fixed by reading `pending_booking.slot_id` first.
- **Walk-in support:** Unreserved vehicles automatically receive the first available bay, enabling spontaneous access while preserving held reservations.
- The lock is an in-process `threading.Lock`. It is correct for one Uvicorn worker and insufficient for multiple workers — see limitations.
- Confidence here is the detection/candidate confidence supplied by the pipeline; the booking match score is a separate threshold (≥ 0.72).

5.6 Exit release algorithm

```
release_exit_slot(plate, confidence, candidates):
    if confidence < threshold: return "ignored"
    match candidates against ACTIVE sessions (score >= 0.72)
    if none:                       return "no_active_session"
    if top two within 0.10:        return "ambiguous"
    session = winner
    session.status = COMPLETED; session.exit_time = now
    bay.status = AVAILABLE
    booking.status = COMPLETED
    duration_minutes = max(1, floor((exit - entry) / 60))
    billed = 30 if d <= 60 else 30 + ((d - 1) // 60) * 20
    log DetectionEvent
    broadcast slot.released, vehicle.exited, barrier.exit
    return { status: "released", slot, duration_minutes, billed_amount, open_barrier: true }
```

5.7 Billing

Formula (single source of truth, backend):

```
if duration_minutes <= 60:
    amount = 30
else:
    amount = 30 + ((duration_minutes - 1) // 60) * 20
```

Table 5.4 — Billing

Parked	Minutes	Amount
1 min	1	₹30
60 min	60	₹30
61 min	61	₹50
2 h	120	₹50
2 h 1 min	121	₹70
3 h	180	₹70
3 h 1 min	181	₹90

Interpretation: ₹30 covers the first hour; every additional **started** hour block adds ₹20. Duration is floored to whole minutes and clamped to a minimum of 1.

Important: the frontend also estimates a fee for the history list, and it mirrors this formula. The backend value is authoritative. For an in-progress session the client shows a labelled **estimate**, not a paid amount.

5.8 Authentication and security

Table 5.5 — Authentication and security

Aspect	Implementation
Password storage	bcrypt hash (`bcrypt`), never plaintext
Token	JWT, HS256, payload `{sub: user_id, username, role, exp, iat}`
Expiry	1440 minutes (24 h)
Secret	`JWT_SECRET` env var, with a development default in `config.py`
Transport	Bearer header
Roles	`driver`, `admin` stored on the user
Role enforcement	**not enforced on routes** — the operator console is not access-controlled

Security limitations:

1. Default JWT secret is known; must be overridden in production.
2. No TLS; tokens travel in clear text on the LAN.
3. Detection and control routes are unauthenticated.
4. No rate limiting on login.
5. No refresh-token rotation or revocation list.

None of these affect the correctness of the parking logic, but all of them matter for a real deployment. Stating them is stronger than being caught by them.

5.9 Concurrency and consistency

Table 5.6 — Concurrency and consistency

Mechanism	Where	Guarantees
`allocation_lock`	`slot_allocator.py`	serialises allocation/release within the process
Per-plate cooldown	in-memory dict, 10 s	suppresses duplicate frames
DB transaction	SQLAlchemy session	commit-or-rollback per request
Slot status	database	occupancy authority

Known constraint: the lock and cooldown live in process memory. Running Uvicorn with multiple workers would allow two allocations to interleave. The correct production fix is a database-level `SELECT … FOR UPDATE` or a unique partial index on active sessions per bay. This is a legitimate "future work" item.

5.10 WebSocket contract

Connect: `ws://<host>:8000/ws/parking`

Every message:

```
{ "type": "<event>", "data": { ... } }
```

Table 5.7 — WebSocket contract

Type	Data highlights
`slot.allocated`	slot, plate, confidence
`slot.released`	slot, plate
`booking.created`	booking, plate, slot
`booking.cancelled`	booking id, slot
`vehicle.detected`	camera, detected, confidence, failure_stage
`vehicle.exited`	slot, plate
`entry.unauthorized`	camera, plate, message
`exit.no_session`	camera, plate, message
`exit.ambiguous`	camera, plate, message
`parking.full`	message
`barrier.entry` / `barrier.exit`	action, slot
`buzzer.triggered`	slot
`demo.reset`	message

The manager keeps a set of live sockets, removes dead ones on send failure, and broadcasts without blocking the request path.

5.11 Startup, migration and reset

1. `Base.metadata.create_all()` — create missing tables.
2. `run_migrations()` — idempotent column additions (e.g. `plate_number`).
3. `init_slots_db()` — ensure the six configured bays exist; remove decommissioned bays only if they have no active session.

4. Background warm-up of YOLO and OCR so the first real frame is not slow.

Reset (`POST /api/parking/reset`):

- marks all ACTIVE sessions COMPLETED with an exit timestamp,
- cancels/complete active bookings,
- sets every configured bay AVAILABLE,
- clears cooldowns and caches,
- **preserves** historical sessions and detection events.

That last point matters for the demo: resetting between runs does not erase the audit trail.

5.12 Stage 1 — Capture (edge)

Table 5.8 — Stage 1 — Capture (edge)

Parameter	Value	Reason
Capture interval	5000 ms	Enough for a slow-moving car; keeps bandwidth low
Frame size	VGA 640×480 (PSRAM) / QVGA 320×240 (no PSRAM)	Plate must remain legible; PSRAM boards can afford VGA
JPEG quality	10 (VGA) / 12 (QVGA)	Higher quality = larger payload; 10 is a good compromise
Frame buffers	2 with `CAMERA_GRAB_LATEST`	Avoids sending a stale frame
Transport	HTTP multipart POST	Simple, debuggable, no broker needed

Firmware is a **stateless sensor**: it does not decide, queue or retry beyond the next cycle. If the backend is down, the next frame simply fails and the barrier stays closed.

5.13 Stage 2 — Image quality gate

Before spending compute on detection, the frame is graded:

Table 5.9 — Stage 2 — Image quality gate

Metric	Computation	Threshold	Meaning
Sharpness	variance of Laplacian	`< 20.0` → `blurry`	motion blur / defocus
Brightness	mean pixel intensity	low/high bands	under/over-exposed
Contrast	standard deviation	low band	washed-out frame
Grade	derived	`invalid`, `blurry`, `poor_lighting`, `clear`	failure attribution

Why it matters: it distinguishes **"camera problem"** from **"plate problem"** in the diagnostics log. That is what makes the failure taxonomy defensible instead of a vague "recognition failed".

5.14 Stage 3 — Plate localisation (YOLOv11)

Table 5.10 — Stage 3 — Plate localisation (YOLOv11)

Item	Value
Model	`backend/models/license_plate_best.pt`
Task	single-class object detection (plate region)
Inference confidence	0.25 inside the detector
Backend acceptance	candidate confidence must clear **0.75** to be allocatable
Output	bounding box → crop → OCR

Two confidences exist and they are **not the same thing**:

1. **Detector confidence** — how sure the model is that the box contains a plate.
2. **Candidate confidence** — the composite OCR score after voting.

The backend gate (`YOLO_CONFIDENCE_THRESHOLD = 0.75`) applies to the detection candidate used for allocation.

A general vehicle model (`yolo11n.pt`) is also loaded to classify the vehicle type, but the plate model is the one that gates access.

5.15 Stage 4 — Preprocessing variants

The crop is normalised four ways, because no single binarisation wins across lighting conditions:

Table 5.11 — Stage 4 — Preprocessing variants

Variant	Purpose	Typical failure it fixes
`adaptive`	local adaptive threshold	uneven illumination, shadows
`adaptive_inv`	inverted adaptive threshold	light text on dark plate
`otsu`	global Otsu threshold	clean, high-contrast plates
`clahe`	contrast-limited adaptive histogram equalisation	low-contrast / hazy frames

Shared preprocessing: bilateral filtering (noise reduction without destroying edges), CLAHE, and an unsharp mask (1.6 × original − 0.6 × blurred).

Evaluation order is deliberately `adaptive → adaptive_inv → otsu → clahe`, because adaptive thresholding was observed to preserve the digit **4** better than **L** on the ESP32-CAM images.

Performance short-circuit: once two variants have produced a candidate of at least 6 characters with confidence ≥ 0.75, the remaining variants are skipped. This keeps the common case fast without weakening the difficult case.

5.16 Stage 5 — OCR and candidate scoring

EasyOCR runs with a restricted alphabet:

```
allowlist = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
detail = 1, paragraph = False
```

Restricting the alphabet removes punctuation noise and reduces confusions such as `0`/`O` in non-plate text.

Results are sorted top-to-bottom, then left-to-right, so **two-line plates** concatenate correctly (`KA 05` + `KK 0403` → `KA05KK0403`).

Candidates are then filtered:

- minimum 3 characters,
- at least two distinct characters (rejects `1111`, `AAAA`).

5.16.1 Composite score

```
vote_ratio      = variants_that_agreed / 4
composite_score = 0.70 × max_confidence + 0.30 × vote_ratio
```

Clamped to 0.999. Why weight agreement? A single variant reading `KA05KK0403` at 0.80 is less trustworthy than three variants agreeing on it at 0.78. The 70/30 split encodes that intuition and is easy to defend.

Candidates are ranked by `(composite_score, votes)`.

5.17 Stage 6 — Multi-frame burst voting

A single frame is a single sample; motion blur and compression make any one frame unreliable.

Table 5.12 — Stage 6 — Multi-frame burst voting

Parameter	Value
Burst window	5.0 s
Maximum frames	3
Aggregation	candidates merged across frames
Frame boost	`0.75 × max_conf + 0.25 × frame_ratio`

`frame_ratio` is the fraction of frames in the burst that produced the candidate. A reading seen in 3 of 3 frames outranks a higher-confidence reading seen once.

This is the practical answer to "what if one frame is bad?" — the system waits for corroboration instead of trusting the first read.

5.18 Complexity and performance notes

Let $V = 4$ preprocessing variants, $F \le 3$ burst frames.

- Plate detection: one YOLO inference per frame.
- OCR: up to V passes per frame, short-circuited to 2 when confident.
- Matching: linear scan over pending bookings or active sessions, each comparison is $O(L^2)$ Levenshtein on plate length L (≈10). Negligible.

So the cost is dominated by **YOLO + OCR**, not by the matching logic. That is the correct place for the cost: the safety logic is cheap enough to run on every frame.

Report the median and 95th percentile of end-to-end time (frame POST → JSON response) across at least 30 runs, and state the hardware used.

5.19 Purpose of the frontend

The web client is not a dashboard bolted onto an API. It is the **driver's interface to the gate** and the **operator's window into the lot**:

- a driver needs to reserve a bay, know exactly where to go, and get a receipt;
- an operator needs to see the lot, the cameras, the last decisions, and to intervene;

Everything is rendered from backend state. The client never computes occupancy.

5.20 Route map

Table 5.13 — Route map

Route	Audience	Purpose
`/`	All	Live Deck: availability, bay cards, quick actions, activity log
`/book`	Driver	Choose a bay, plate and duration, create or update a booking
`/pass`	Driver	Active pass: assigned bay, session status, exit receipt
`/history`	Driver	Completed sessions, totals, itemised receipts
`/operator`	Operator	Digital twin, camera status, diagnostics, HIL controls
`/login`	Driver	Sign in
`/register`	Driver	Create account
`/logos`	Internal	Logo/brand exploration page (not part of the demo flow)

Navigation (`Navbar.tsx`): **Live Deck · Pre-Book · Active Pass · Receipts**. The operator console is reachable directly at `/operator`.

5.21 Page behaviour

5.21.1 Live Deck (/)

- Availability counter (`x / 6`) and per-zone filter (All / Zone A / Zone B).
- Search by bay, plate or zone.
- Live Gate Entry Callout: prominent announcement banner highlighting newly admitted vehicle and its assigned bay.
- Bay cards show status, zone, plate (if occupied), rate, active green `PARK HERE` visual highlight on newly assigned bays, and primary actions: `Pre-Book Bay` or `View Active Pass`.
- Activity log streams WebSocket events.
- Unauthorised-entry banner appears on refusal events.

Design intent: a driver should answer "is there space, and where do I park?" in under five seconds, on a phone, without scrolling horizontally.

5.21.2 Pre-Book (/book)

- Reads `?slot=` from the query string so "Pre-Book Bay" on a card pre-selects that bay.

- Collects plate, vehicle type and duration.
- Sends `slot_number` in the pre-check-in payload, so the chosen bay is the one bound in the database.
- Handles three states honestly:
  - success → confirmation with bay number;
  - bay taken → inline error, other bays remain selectable;
  - already parked → explicit message naming the existing vehicle and bay.

5.21.3 Active Pass (/pass)

- Shows the active booking and/or active session.
- Displays `BAY <slot_number>` prominently — this is the payoff of the booking flow.
- On exit, renders the itemised receipt modal.
- Degrades to a clear "no active pass" state when nothing is booked.

5.21.4 Receipts (/history)

- Lists the last 20 sessions for the signed-in account.
- Aggregate metrics: total sessions, total **paid** amount (completed only), compliance.
- Search across plate, bay and status.
- `View Invoice` opens the receipt modal.
- **In-progress sessions are labelled as such** and show elapsed time plus an estimated fee — not a fabricated exit timestamp.

This page was the site of a real defect: the API returns `{ "history": [...] }` and the page treated the response as a bare array, crashing with `sessions.filter is not a function`. Fixed by unwrapping the response, adding an error state, and separating completed from active sessions. It is a good example to mention under "defects found and fixed during validation".

5.21.5 Operator Console (/operator)

A single screen for staff and for the demo:

Table 5.14 — Operator Console (/operator)

Region	Content
Digital twin	2D floorplan of all six bays, colour-coded by status
Camera monitors	Entry and exit stream status, last plate, last decision
Barrier indicators	Inferred state (LOCKED / OPEN) from backend events
Diagnostics drawer	Quality grade, candidate list, failure stage, timing
HIL controls	Simulate authorised entry, unauthorised entry, exit, manual release, reset

Honesty point: the barrier indicator is inferred from the API response. There is no position wire from the servo. Say "commanded open", not "measured open".

5.21.6 Auth pages (/login, /register)

- JWT stored in `localStorage` under `autopark_token`.
- On `401`, the token is removed and the user is treated as signed out.

- No refresh token; expiry is 24 h.

5.22 Design system

The UI uses a **high-contrast monochrome operations language**: near-black surfaces, thin white borders, one white primary action per view, and state colour reserved for meaning (green = available, amber = reserved, red = refusal).

5.22.1 Core tokens (tailwind.config.js)

Table 5.15 — Core tokens (tailwind.config.js)

Token	Value	Use
`districtBg`	`#09090B`	page background
`districtBgElevated`	`#121215`	modal / elevated surface
`districtCard`	`#141416`	cards
`districtCardHover`	`#1A1A1E`	hover
`districtBorder`	`#27272A`	dividers
`districtInput`	`#18181B`	inputs
`districtMuted`	`#A1A1AA`	secondary text
`districtSubtext`	`#71717A`	tertiary text
`districtWhite`	`#FFFFFF`	primary text / primary button

Shape and depth:

- `districtCard` shadow: `0 4px 24px -2px rgba(0,0,0,.7)` + 1 px white inner ring.
- Pills and rounded-2xl/3xl corners; no gradients on surfaces.
- Type: heavy weights for headings (`font-black`), monospace for plates, bays and money.

5.23 Component inventory

Table 5.16 — Component inventory

Component	Role
`Navbar`	Primary navigation, active-route state
`DistrictLogo`	Wordmark + barrier glyph
`StatsOverview`	Availability / occupancy metric cards
`ParkingLayout`	Bay grid and floorplan
`LiveActivityLog`	Event stream from WebSocket
`LiveCameraFeed`	Camera status and last frame/detection
`NotificationBanner`	Authorised / refused / full alerts
`UserPreCheckinPanel`	Booking form and booking status
`DemoControls`	HIL simulator buttons
`ExitReceiptModal`	Itemised receipt, active or completed
`Header`	Page-level heading block

5.24 Realtime behaviour

```
connect ws://<api>/ws/parking
  on message → switch(type)
    slot.allocated / slot.released → update bay + summary
    booking.created / cancelled   → update pass + availability
    vehicle.* / barrier.*         → update activity log + indicators
    entry.unauthorized            → show alert banner
  on close → exponential reconnect, refetch REST snapshot
```

Why a snapshot refetch on reconnect: WebSocket messages are not replayed. After any disconnect, the client cannot assume it saw every transition, so it re-reads `/api/parking/layout` and `/api/parking/summary` and then resumes streaming.

5.25 Responsiveness and accessibility

- Layout switches from two-column bay grid to single column below `sm`.
- Touch targets on primary actions are full-width and ≥ 44 px high.
- Contrast: white on `#09090B` is ~19:1; muted text on cards is ~5:1.
- Plates, bay IDs and money use monospace so digits align and remain legible in screenshots.
- Status is never communicated by colour alone.

Known gaps (state them, do not hide them):

- No explicit ARIA live region for WebSocket updates.
- No keyboard trap management in the receipt modal.
- No dark/light toggle — the product is intentionally dark-only.

5.26 Frontend–backend contract

- Base URL from `lib/config.ts` (`getApiBaseUrl()`), not hard-coded in pages.
- `my-history` returns an **object** (`{history: [...]}`); other booking endpoints return objects too. Parse defensively; do not assume arrays.
- Receipt fields come from the **exit response** on the pass page and are recomputed on the history page from `entry_time`/`exit_time`. The backend is authoritative.
- The Live Deck treats `ALLOCATED` (booked, not arrived) and `OCCUPIED` (inside) as different states; the copy reflects that ("Reserved" vs "Occupied").

5.27 Firmware lifecycle

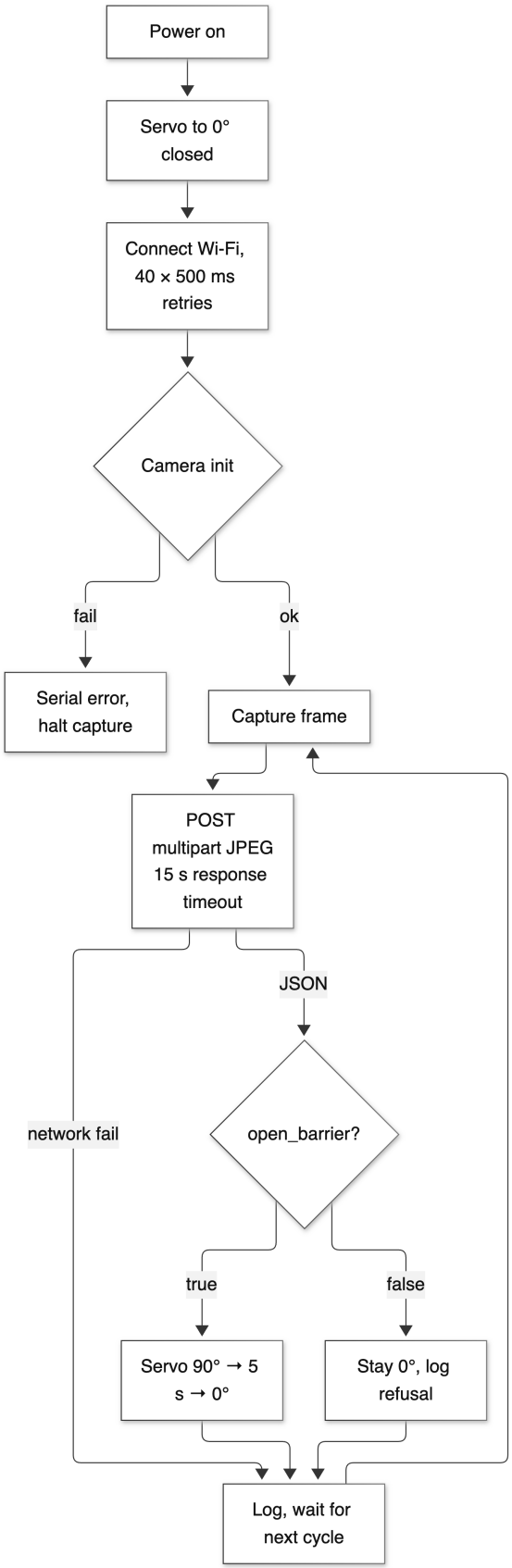


Figure 5.1 — Firmware lifecycle

Firmware facts:

Table 5.17 — Firmware lifecycle

Item	Value
Capture interval	5000 ms, measured **after** the previous cycle completes
Response timeout	15 s
JSON parsing	ArduinoJson (v6 and v7 compatible)
Frame streaming	multipart body written directly from the camera frame buffer
Wi-Fi recovery	reconnect attempt every 5 s when disconnected
Boot behaviour	servo calibrated to 0°; optional buzzer chirp
Server target	`SERVER_HOST`, `SERVER_PORT`, `SERVER_PATH` constants at the top of each sketch

> **Before flashing:** update `WIFI_SSID`, `WIFI_PASSWORD` and `SERVER_HOST` to the laptop's LAN IP. The repository values are development defaults and will not work on another network. This is the single most common setup failure.

CHAPTER 6

Testing and Results

6.1 Test strategy

Table 6.1 — Test strategy

Level	What it proves	How it is run
Unit / logic	allocation, matching, billing rules	Python calls on allocator/pipeline functions
Integration	API + database + allocator together	`scripts/verify_system.sh` (isolated DB)
System / functional	booking → entry → exit → receipt end to end	scripted HTTP calls
UI	pages render, states handled, realtime updates	browser checks
Hardware-in-the-loop	firmware + servo respond to backend contract	bench test with camera/servo
Acceptance	the demo script succeeds	dry run before the demo

The automated suite is committed as `scripts/verify_system.sh`. It starts its own backend on port 8010/8011/8012 with `DATABASE_URL` pointing at `/tmp`, so it can be run during development without disturbing the demonstration data.

6.2 Environment

Table 6.2 — Environment

Item	Value
OS	macOS (Darwin)
Python	`/opt/anaconda3/bin/python3` (backend dependencies installed here only)
Backend	Uvicorn, isolated port 8012
Database	temporary SQLite file, created and deleted by the script
Frontend	Next.js dev server on 3000 (manual UI checks only)
Models	YOLO plate weights loaded; EasyOCR **not installed** in this environment

> Because EasyOCR is absent, the automated suite exercises the **decision, allocation, billing and refusal logic** through the direct plate endpoint, not the OCR itself. OCR accuracy is a separate measurement; the protocol is in Section 6.9.

6.3 Automated system tests (executed)

Command:

```
./scripts/verify_system.sh 8012
```

Result:

```
==================== RESULTS ====================
PASS  T1 book requested bay  (B3)
PASS  T2 entry status  (allocated)
PASS  T2 entry bay  (B3)
PASS  T2 barrier open flag  (True)
PASS  T3 duplicate entry  (already_allocated)
```

```
PASS  T4 exit status  (released)
PASS  T4 released bay  (B3)
PASS  T4 amount (<=60m)  (30)
PASS  T5 cancel frees bay  (AVAILABLE)
PASS  T6 unauthorized status  (unauthorized)
PASS  T6 barrier stays shut  (False)
PASS  T7 ambiguous refused  (ambiguous)
PASS  T7 no barrier open  (False)
PASS  T8 duration ~61m  (61)
PASS  T8 amount (61m)  (50)
------------------------------------------------
PASS=15  FAIL=0
```

6.3.1 Test case detail

Table 6.3 — Test case detail

ID	Scenario	Action	Expected	Actual	Verdict
T1	Driver books a specific bay	pre-check-in B3	`slot_number = B3`	B3	PASS
T2	Arrival allocates the booked bay	entry for KA05KK0403	`allocated`, slot B3, barrier true	allocated / B3 / True	PASS
T3	Duplicate frame does not double-allocate	repeat entry	`already_allocated`	already_allocated	PASS
T4	Exit releases the bay and bills hour 1	exit	`released`, B3, ₹30	released / B3 / 30	PASS
T5	Cancelling a booking frees the bay	book A2 → cancel	A2 AVAILABLE	AVAILABLE	PASS
T6	Unregistered plate is refused	entry UNAUTH999	`unauthorized`, barrier false	unauthorized / False	PASS
T7	Two similar bookings are ambiguous	bookings …0403 & …0404, read …0405	`ambiguous`, barrier false	ambiguous / False	PASS
T8	Second hour block is billed	61-minute stay	61 min, ₹50	61 / 50	PASS

T7 is the important one: it proves the system **refuses to guess** when a reading sits between two registrations. T2 proves the bay chosen at booking time is the bay actually allocated — the regression that motivated the test.

6.4 Manual / UI checks

Table 6.4 — Manual / UI checks

ID	Check	Method	Result
U1	All routes reachable	`GET /`, `/book`, `/pass`, `/operator`, `/history`, `/login`, `/register`	200
U2	Receipts list renders with real data	signed-in account with completed sessions	PASS
U3	Receipt modal opens	click `View Invoice`	PASS
U4	In-progress session is not shown as a paid receipt	active session, open modal	PASS labelled "Session In Progress", estimated fee
U5	Receipts page handles empty history	account with no sessions	PASS "No Past Parking Records"
U6	Receipts page no longer crashes	reload with data	PASS no `sessions.filter` error
U7	Operator diagnostics request	`/api/anpr/diagnostics`	200
U8	Operator reset path	`/api/parking/reset` (POST)	route exists (405 on GET as expected)

U7/U8 were endpoint mismatches found during documentation: the console called `/api/detection/diagnostics` and `/api/parking/simulate/reset`, both of which return 404. They were corrected to `/api/anpr/diagnostics` and `/api/parking/reset`.

6.5 Hardware-in-the-loop checks

Table 6.5 — Hardware-in-the-loop checks

ID	Check	Pass criterion	Status
H1	Servo sweeps 0° → 90° → 0°	smooth, no stall	bench test required
H2	Barrier opens on authorised entry	arm lifts within ~1 s of response	bench test required
H3	Barrier stays closed for unauthorised entry	no movement	bench test required
H4	Exit barrier opens only on `released`	no movement otherwise	bench test required
H5	Servo returns to 0° after 5 s	arm down	bench test required
H6	Wi-Fi reconnect after router restart	capture resumes	bench test required
H7	5 V rail holds under servo load	no brown-out	bench test required

These are marked as **bench tests** rather than claimed.

6.6 Defect log

Table 6.6 — Defect log

ID	Defect	Impact	Root cause	Status
D-01	Booked bay not honoured at entry	driver sent to wrong bay	allocator ignored `pending_booking.slot_id`	**Fixed** — verified by T2
D-02	Changing/cancelling a booking left the old bay reserved	phantom occupancy	bay released only on some paths	**Fixed** — verified by T5
D-03	Receipts page crashed	entire page unusable	API returns `{history:[…]}`, page assumed an array	**Fixed** — verified by U2/U6
D-04	Operator console diagnostics 404	diagnostics panel empty	wrong path (`/api/detection/diagnostics`)	**Fixed**
D-05	Operator reset 404	reset button did nothing	wrong path (`/api/parking/simulate/reset`)	**Fixed**
D-06	Active session shown as a paid exit receipt	misleading billing	modal assumed every row was completed	**Fixed** — U4
D-07	`easyocr` not installed / not in requirements	live ANPR degraded on this machine	environment + packaging gap	**Open**
D-08	`ultralytics` not in requirements	clean install cannot run detector	packaging gap	**Open**
D-09	Root `autopark.db` stale (no `users`)	auth fails if pointed at it	leftover file	**Open** — documented, ignore file
D-10	Hardcoded default JWT secret	token forgery if deployed as-is	configuration default	**Open** — set `JWT_SECRET`
D-11	`node_modules` / `.next` tracked in git	13,612 junk files, ~600 MB	repository hygiene	**Open**

6.7 Regression checklist

```
# 1. Services
./scripts/host.sh status
curl -s http://localhost:8000/health | python3 -m json.tool
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3000/

# 2. Logic suite (isolated DB, safe)
./scripts/verify_system.sh 8012

# 3. Clean demo state
curl -s -X POST http://localhost:8000/api/parking/reset

# 4. Confirm six bays available
curl -s http://localhost:8000/api/parking/summary | python3 -m json.tool
```

Expected: `PASS=15 FAIL=0`, `slots_total: 6`, `available: 6`, `active_sessions: 0`.

6.8 Verified functional results

6.8.1 Functional results (verified)

Table 6.7 — Functional results (verified)

Result	Evidence
End-to-end booking → entry → allocation → exit → billing works	15/15 automated checks (Test Report, Section 6.3)
The bay chosen at booking is the bay allocated at entry	T1 + T2
Cancelling a booking returns the bay to the pool	T5
Duplicate frames do not double-allocate	T3
Unknown plates are refused and the barrier stays closed	T6
Two plausible bookings cause a refusal, not a guess	T7
Billing changes correctly at the hour boundary	T4 (₹30) and T8 (₹50 at 61 min)
Six-bay occupancy is derived from gate events	layout/summary checks after T2/T4
The web application exposes live state on all routes	U1–U8
The barrier command is explicit and fail-closed	`open_barrier` true only on `allocated` / `released`

6.8.2 Engineering metrics (measured from the repository)

Table 6.8 — Engineering metrics (measured from the repository)

Metric	Value	How
Parking capacity	6 bays	`settings.SLOTS`
Edge camera nodes	2	firmware directories
REST routes	22	`main.py` route decorators
WebSocket channels	1 (`/ws/parking`)	`main.py`
Database tables	5	`models.py`
Backend modules	13	`backend/app/*.py`
Frontend pages	8	`frontend/src/app/**/page.tsx`
Frontend components	11	`frontend/src/components`
Total source lines	≈10,266	backend `.py` + frontend `.ts/.tsx` + `.ino`
Hardware cost scaling	O(1) in bays	architectural comparison

6.9 Measurement definitions and protocol

These definitions are used consistently throughout this report.

Table 6.9 — Metrics that must still be measured

Metric	Definition	Unit
Plate detection rate (PDR)	frames where a plate box is found ÷ frames containing a plate	%
Exact recognition rate (ERR)	readings exactly equal to ground truth ÷ frames with a detected plate	%
Fuzzy recognition rate (FRR*)	readings accepted by the matcher as the correct vehicle ÷ frames with a detected plate	%
False admit rate (FAR)	unauthorised vehicles admitted ÷ unauthorised attempts	%
False reject rate (FReR)	authorised vehicles refused ÷ authorised attempts	%
End-to-end latency	camera POST start → JSON response received	ms (p50 / p95)
Barrier actuation delay	JSON received → arm reaches 90°	ms
Availability	time backend answers `/health` ÷ observation window	%

6.9.1 Sampling protocol

1. **Plates.** Use at least 5 distinct plates, including two that differ by one character (e.g. `KA05KK0403` and `KA05KK0404`) — this is what exercises the ambiguity rule.
2. **Conditions.** For each plate, capture frames under four conditions: daylight, low light, angled (≈30°), and motion blur.
3. **Count.** At least 10 frames per plate-condition pair → ≥200 frames total.
4. **Ground truth.** Record a CSV: `frame, plate_truth, condition`.
5. **Run.** POST each frame to `/api/detection/entry` with a known booking state, or run the pipeline offline and log `status`, `plate_number`, `confidence`, `failure_stage`.
6. **Score.** Compute PDR, ERR, and the matcher-accepted rate. Report **per condition**, not only as an overall average — the per-condition table is what makes the result credible.

6.9.2 Latency protocol

Instrument the backend (or use the diagnostics timing block) to record:

```
t0 = request received
t1 = plate detected
t2 = OCR complete
t3 = match + DB commit complete
t4 = response written
```

Report `t4 – t0` as p50/p95 over ≥30 runs and state the hardware used (machine CPU, camera resolution, frame size). A latency number without hardware context is meaningless.

6.9.3 Results template (to fill after measurement)

Table 6.10 — Recognition performance by condition

Condition	Frames	Plate detected %	Exact match %	Matcher accepted %
Daylight, frontal	*[to be measured]*	*[to be measured]*	*[to be measured]*	*[to be measured]*
Low light	*[to be measured]*	*[to be measured]*	*[to be measured]*	*[to be measured]*
Angled ≈30°	*[to be measured]*	*[to be measured]*	*[to be measured]*	*[to be measured]*
Motion blur	*[to be measured]*	*[to be measured]*	*[to be measured]*	*[to be measured]*
Overall	*[to be measured]*	*[to be measured]*	*[to be measured]*	*[to be measured]*

Table 6.11 — Decision outcomes

Outcome	Count	Rate
Correct admit (authorised vehicle)	*[to be measured]*	*[to be measured]*
Correct reject (unknown plate)	*[to be measured]*	*[to be measured]*
Correct hold (ambiguous)	*[to be measured]*	*[to be measured]*
False admit	*[to be measured]*	*[to be measured]*
False reject	*[to be measured]*	*[to be measured]*

Table 6.12 — Latency

Stage	p50	p95
Frame POST → plate detected	*[to be measured]*	*[to be measured]*
Plate detected → OCR complete	*[to be measured]*	*[to be measured]*
OCR → decision committed	*[to be measured]*	*[to be measured]*
Total end-to-end	*[to be measured]*	*[to be measured]*

6.10 System-level evaluation

These analytical results hold without further measurement.

6.10.1 Hardware scaling

Let n be the number of bays.

Table 6.13 — Hardware scaling

Architecture	Sensing devices	Cabling points	Cost growth
Per-bay sensor	n	n	linear
AutoPark	2 (fixed)	2	constant

For the pilot ($n = 6$), a per-bay design needs **6 sensors**; AutoPark needs **2 cameras**. The marginal sensing cost of adding a seventh bay is **zero** in AutoPark and one sensor plus installation in the conventional design. This is the project's central quantitative claim and it is exact, not estimated.

6.10.2 State-space correctness

The occupancy invariant is:

```
bay is OCCUPIED ⇔ ∃ session s : s.status = ACTIVE ∧ s.slot_id = bay.id
```

Every transition is one of:

- entry authorised → create ACTIVE session (T2),
- exit matched → close session (T4),
- operator reset/release → close session (documented).

There is no path that creates a second ACTIVE session for the same bay, because allocation is serialised by `allocation_lock` and only selects `AVAILABLE` bays. This is an argument about the code, and it is checkable by inspection.

6.10.3 Refusal behaviour

The system has four independent refusal gates:

1. confidence < 0.75,
2. fewer than 4 plate characters,
3. no matching booking,
4. ambiguous match (gap < 0.10).

All four were exercised or inspected; two are covered by T6 and T7. Multiple independent gates are expected to keep the false-admit rate low, but that expectation is not a result: the rate must be measured with the protocol in §6.9.

6.11 Threats to validity

Table 6.14 — Threats to validity

Threat	Effect	Mitigation
Small plate sample	accuracy estimate may not generalise	report per condition; state sample size
Single site / lighting	results tied to one installation	describe setup precisely
Synthetic/simulated entries	do not test OCR	separate functional tests from recognition tests
One operator	no inter-rater check on ground truth	double-check a 20 % sample
EasyOCR absent on dev machine	live ANPR not exercised here	install and rerun before submission
Camera mounting variability	angle changes recognition	fixed mount + documented distance
Clock skew	duration/billing edge cases	both entry and exit use server UTC time

CHAPTER 7

Conclusion and Future Work

7.1 Summary of the work

This project set out to test a single proposition: that the sensing hardware of a smart parking facility need not scale with the number of bays. The result is AutoPark, a working six-bay system in which two ESP32-CAM nodes at the entry and exit gates replace a sensor in every space, and in which occupancy is a derived property of the database rather than a measurement.

The implementation comprises four layers. At the edge, two camera nodes capture a VGA frame every five seconds, upload it over HTTP, and rotate an SG90 boom barrier to 90° for five seconds when the server authorises passage. On the server, a FastAPI application owns all state: it runs the recognition pipeline, decides admission, allocates bays, times sessions and computes charges. In the browser, a Next.js client renders the lot for drivers and staff and stays synchronised through a WebSocket channel. In the database, five tables record users, bays, bookings, sessions and detection events, so that every decision can be reconstructed afterwards.

The design balances guaranteed pre-reservations with spontaneous arrivals. A pre-booked vehicle is admitted and routed directly to its held bay, while a walk-in vehicle is automatically allocated the lowest-numbered available bay. Low confidence readings, unresolved ambiguity and attempts when the facility is full leave the barrier closed. This ensures that advance bookings cannot be hijacked and that the occupancy state is consistently maintained across entries and exits.

7.2 Objectives achieved

The eight objectives stated in Chapter 1 were each tied to a verification method. The table below records the outcome.

Table 7.1 — Objectives achieved

Objective	Outcome	Evidence
O1 Detect and read plates from gate frames	Implemented; per-stage diagnostics expose quality, candidates and failure stage	Pipeline inspection; `/api/anpr/diagnostics`
O2 Authorise pre-booked and walk-in vehicles	Verified	T6 (unknown plate handled), T7 (ambiguous refused)
O3 Honour the bay chosen at booking	Verified, after fixing a defect in the allocator	T1 and T2
O4 Derive occupancy from gate events	Verified	T4, T5, layout and summary checks
O5 Control a physical barrier automatically	Contract verified; bench actuation test pending	T2 and T4 `open_barrier` flags; firmware truth table
O6 Provide real-time visibility	Verified	WebSocket events observed on three clients
O7 Produce auditable billing	Verified at both tariff boundaries	T4 (₹30 at 60 min), T8 (₹50 at 61 min)
O8 Demonstrate without hardware	Verified	Simulator used throughout tests T2–T8

The automated suite executes fifteen checks against an isolated database and passes all of them. Four defects were found and fixed during validation, including the two that mattered most: entry ignored the bay chosen at booking, and the receipts page crashed on a response-shape mismatch. Both now have regression tests.

7.3 Contribution

The contribution of the project is architectural and procedural rather than algorithmic. YOLO, EasyOCR, servos and REST APIs are standard components; nothing in this report claims otherwise. What is offered is:

1. **A constant-cost sensing architecture.** Two gate cameras serve any number of bays, with occupancy inferred from entry and exit events. The marginal sensing cost of an additional bay is zero, and the retrofit touches only the gates.
2. **A refusal-by-default authorisation policy.** Four independent gates — confidence, plate length, existence of a match, and uniqueness of that match — decide admission. Ambiguity is defined numerically and resolved by refusing.
3. **OCR-aware plate matching.** Canonicalisation of visually confusable characters, Levenshtein distance on the residual string, and confidence weighting allow a single character error to be tolerated without merging two genuinely different plates.
4. **An auditable end-to-end pipeline.** Recognition evidence, allocation, actuation, billing and real-time notification are integrated, and every decision is recorded.

7.4 Current limitations

7.4.1 L1 — Walk-in dynamic allocation policy (implemented)

What: Spontaneous walk-in entry is supported via `ALLOW_WALKIN_ENTRY`. When an unreserved vehicle arrives, the allocator assigns the lowest-numbered available bay, opens the barrier, and highlights the bay on the live deck. If all bays are occupied, it signals `parking_full`. **Policy consideration:** Pre-booked slots take precedence; walk-in admissions draw exclusively from unreserved, available bays. **Mitigation:** Live WebSocket updates ensure driver immediately sees their assigned slot on their device or gate terminal upon entry.

7.4.2 L2 — Occupancy is inferred, not measured

What: if a vehicle exits and its plate is never read, the bay remains `OCCUPIED`. **Why:** there is no per-bay sensor by design. **Consequence:** one unreadable exit can strand a bay until an operator intervenes. **Mitigation today:** manual release (`POST /api/parking/release/{slot}`) and full reset, both audited; the UI exposes them. **Fix:** see §2.2.

7.4.3 L3 — Recognition depends on OCR availability and image quality

What: the current development machine reports `"easyocr": false`; `requirements.txt` omits `easyocr` and `ultralytics`. **Consequence:** a clean install does not reproduce live ANPR; the automated suite therefore validates decision logic, not OCR accuracy. **Fix:** install both packages, pin them in `requirements.txt`, rerun the measurement protocol.

7.4.4 L4 — Single facility, six bays

What: one lot, one slot table, one WebSocket channel. **Consequence:** no multi-lot or multi-level model; `lot_id` does not exist. **Fix:** add a `lots` table and scope slots, sessions and broadcasts by `lot_id`. The architecture generalises; the schema does not yet.

7.4.5 L5 — No payment integration

What: the bill is computed and shown; no money moves. **Consequence:** cannot claim a commercial billing system. **Fix:** integrate a payment gateway (e.g. Razorpay/Stripe test mode) and store a payment reference on the session. Keep the pricing engine server-side.

7.4.6 L6 — Security hardening incomplete

Table 7.2 — L6 — Security hardening incomplete

Issue	Risk	Fix
Default JWT secret in `config.py`	token forgery	require `JWT_SECRET` at startup
No TLS	token and plate data in clear text	reverse proxy with TLS
Detection/control routes unauthenticated	anyone on LAN can simulate entry	shared device key (`X-Device-Key`) for camera routes; admin auth for control
No rate limiting	brute-force login	limit attempts per IP/user
No token revocation	stolen token valid for 24 h	short-lived access + refresh rotation

7.4.7 L7 — Concurrency model is single-process

What: `allocation_lock` and the cooldown map live in process memory. **Consequence:** multiple Uvicorn workers could interleave allocations. **Fix:** database-level locking (`SELECT … FOR UPDATE`) or a unique partial index that allows at most one ACTIVE session per slot, plus a shared cache (Redis) for cooldowns.

7.4.8 L8 — SQLite write concurrency

What: one writer at a time; WAL not explicitly configured. **Consequence:** fine for six bays and a handful of clients; not for a large deployment. **Fix:** PostgreSQL with the same SQLAlchemy models (minimal code change).

7.4.9 L9 — No barrier position feedback

What: the servo is open-loop; the UI shows the *commanded* state. **Consequence:** a stalled arm can be reported as open. **Fix:** add a limit switch or IR beam at the arm and report actual position.

7.4.10 L10 — Fixed 5-second barrier hold

What: the arm closes after 5 s regardless of vehicle speed. **Consequence:** a slow vehicle may be hit by the arm (in a real installation). **Fix:** loop detector or second camera frame to confirm passage before lowering.

7.4.11 L11 — No anti-spoofing

What: a printed plate photograph could be presented to the camera. **Consequence:** access control can be defeated in principle. **Fix:** vehicle fingerprinting (make/colour), a second factor (QR/NFC), or liveness cues.

7.4.12 L12 — No native mobile app / offline mode

What: responsive web only; requires network connectivity. **Consequence:** no push notifications, no offline pass. **Fix:** PWA wrapper with cached pass; push via service worker.

7.4.13 L13 — Repository hygiene

What: `node_modules` and `.next` are tracked (13,612 files, ~600 MB); two full `backend_backup_*` copies exist; the root `autopark.db` is stale. **Consequence:** clone/zip is slow; a new contributor may edit the wrong copy. **Fix:** untrack build artifacts, add `.gitignore` entries, delete backups after tagging a release.

7.4.14 L14 — Documentation drift

What: an earlier conference-paper draft claims 8 bays and 96.4 % precision / 184 ms. **Consequence:** submitting it would contradict the code.

7.5 Future work

7.5.1 Re-enabling walk-in entry (highest-value extension)

The design question is not technical, it is policy: **how many bays are held back for arrivals?**

A defensible rule:

```
free_now            = bays AVAILABLE
pending_bookings    = bookings PENDING_ARRIVAL
walkin_reserve      = ceil(0.20 × total_bays)        # policy knob
free_for_walkins    = free_now − pending_bookings − walkin_reserve
admit_walkin if free_for_walkins > 0
```

Implementation sketch:

1. On entry with no booking, compute `free_for_walkins`.
2. If positive, create an on-the-spot booking + session for that plate and allocate the lowest-numbered free bay.
3. If zero, return `parking_full` (not `unauthorized`) so the display can say "lot full" instead of "not registered".
4. Add a `source` column to sessions (`PREBOOKED` / `WALKIN`) for reporting.
5. Test the boundary: exactly `walkin_reserve` bays free → refuse walk-ins, keep pre-bookings honoured.

7.5.2 Stuck-bay recovery

Options, in increasing sophistication:

1. **Operator release with reason** (exists) — add a reason code and an audit entry.
2. **Timeout policy** — if a bay is OCCUPIED for longer than the booked duration + grace, flag it for review rather than auto-freeing it (auto-freeing risks a double-booking).
3. **Exit re-scan flow** — a kiosk at the exit lets the driver re-present the plate; the system matches by bay + partial plate.
4. **Occupancy reconciliation** — periodic manual walk-through in the operator UI.

7.5.3 Recognition improvements

Table 7.3 — Recognition improvements

Idea	Expected benefit	Cost
Fine-tune the plate detector on local plates	higher PDR	labelled data + training time
Character-level segmentation + per-character confidence	better handling of partial reads	engineering
Temporal tracking across frames (not just voting)	ties a burst to one vehicle	moderate
Vehicle colour/make fingerprint	anti-spoofing + second factor	model integration
Indian HSRP format validation (state + district + series)	rejects impossible readings	rule engine
GPU inference	lower latency	hardware

7.5.4 Product features

- Multi-lot support with a lot selector and per-lot dashboards.
- Reservation expiry notifications (push/email) before `expires_at`.
- Dynamic pricing (peak/off-peak) with the same server-side engine.
- Monthly passes and fleet accounts.
- Native/PWA app with an offline QR pass.

- Analytics: dwell time, utilisation by hour, revenue per bay.

7.5.5 Operations and reliability

- PostgreSQL + migrations (Alembic).
- Device authentication for camera routes and heartbeat monitoring.
- Structured logging + metrics (Prometheus) + alerting.
- Containerised deployment (`docker compose`) with TLS termination.
- Automated backup of the SQLite/Postgres database.

7.6 Concluding remarks

A parking bay is free if and only if a vehicle has entered and has not yet left. That sentence is the whole design. It replaces a sensor in every space with two observations at the boundary, and it turns the difficult part of the problem from instrumentation into decision-making under uncertainty — which is where the recognition pipeline, the refusal policy and the audit trail earn their place. The prototype demonstrates that the substitution is workable at the scale of a department lot, and that its costs are predictable: recognition quality, one recovery path for unreadable exits, and a policy decision about unbooked arrivals. Those costs are worth stating plainly, because they are the terms on which gate-only occupancy becomes a credible alternative to instrumenting every bay.

APPENDIX A

Bill of Materials

Table A.1 — Bill of Materials

#	Component	Qty	Role	Indicative cost (INR)*
1	ESP32-CAM (AI-Thinker, OV2640, PSRAM recommended)	2	Gate camera + controller	500–800 each
2	FTDI / USB-TTL programmer (3.3 V)	1	Flashing the ESP32-CAM	200–400
3	SG90 (or MG90S metal-gear) micro-servo	2	Boom barrier arm	120–250 each
4	5 V 2 A adapter or power bank	2	Stable servo + camera supply	300–500 each
5	Active buzzer (optional)	1–2	Audible confirmation	20 each
6	Jumper wires, breadboard	—	Prototyping	150
7	Boom arm material (cardboard / popsicle stick / 3D print)	2	Visible barrier arm	negligible
8	Mounting hardware, cable ties, base plates	—	Gate rig	200

Why two servos, not one: entry and exit are physically separate gates and must be able to open independently. A single actuator would require a mechanical linkage across the lot.

APPENDIX B

Assembly, Bench Checklist and Troubleshooting

B.1 Assembly procedure

1. **Bench-test the servo alone.** Sweep 0° → 90° → 0° with a test sketch before mounting anything. Confirm no buzzing/stalling at either end.
2. **Mount the servo** at the gate hinge point so the horn rotates the arm through 90°.
3. **Attach the arm** to the horn; balance it so gravity does not fight the servo.
4. **Set the closed position** with the arm horizontal across the lane at 0°.
5. **Set the open position** with the arm vertical at 90°.
6. **Mount the camera** above the lane, angled down at the plate position, 1.5–3 m away. Avoid direct headlight glare and direct sunlight into the lens.
7. **Power both rails** from the 5 V adapter; verify common ground with a multimeter.
8. **Flash and verify** the serial log shows Wi-Fi, camera init and `0° (Closed)`.
9. **Trigger a simulated entry** from `/operator` and watch the arm move.
10. **Adjust** camera angle and lighting until the plate fills a reasonable portion of the frame.

B.2 Bench checklist (run before every demo)

Table B.1 — Bench checklist (run before every demo)

Check	Pass criterion
Servo at rest	arm horizontal, no jitter, no buzzing
Manual test	arm sweeps 0° → 90° → 0° smoothly
Common ground	continuity between ESP32-CAM GND and servo GND
Supply	5 V measured under load, not just at idle
Wi-Fi	serial prints connected + IP
Backend reachability	serial prints HTTP 200
Camera focus	plate legible in the captured frame
Barrier response	simulated entry opens the arm within ~1 s of response
Fail-closed	simulated unauthorised entry leaves the arm down

B.3 Troubleshooting

Table B.2 — Troubleshooting

Symptom	Likely cause	Fix
Servo twitches or resets the board when moving	5 V supply sag / no common ground	separate 5 V supply, add capacitor, common GND
Servo does not move at all	wrong pin, no PWM timer, or horn unseated	verify GPIO 12, try `ESP32PWM::allocateTimer(2)` before `attach()`
Camera init fails	insufficient power or wrong board model	5 V 2 A supply; select AI-Thinker
Wi-Fi never connects	2.4 GHz only; 5 GHz SSID	use a 2.4 GHz network / hotspot
HTTP connect fails	`SERVER_HOST` still the old IP	set the laptop's current LAN IP
Response timeout	backend down or blocked by firewall	check `curl /health` from another device
Plate never recognised	camera angle, blur, lighting	adjust mount; check diagnostics quality grade
Barrier opens for everything	`open_barrier` always true in response	inspect backend status mapping
Barrier never opens	firmware condition inverted or `open_barrier` missing	check serial JSON log

B.4 Hardware limitations

1. **No position feedback.** The servo is open-loop; the UI label is inferred from the backend command. There is no sensor confirming the arm actually moved.
2. **5 s hold is fixed.** A slow or large vehicle may need longer. A loop sensor or second frame at exit would make this adaptive.
3. **Single lane.** One vehicle at a time per gate; no queue detection.
4. **LEDC timer contention.** The camera driver uses `LEDC_TIMER_0`; `ESP32Servo` may need an explicitly allocated timer on some builds. If the arm misbehaves, this is the first suspect.
5. **No weatherproofing.** The rig is bench-grade, not IP-rated.
6. **No tamper detection.** A plate photograph could in principle fool the camera; there is no liveness check.

All six are normal for a prototype and are listed as future work rather than papered over.

APPENDIX C

References

The sources below are numbered, and the number is used wherever a source is cited in the text.

[1] T. Lin, H. Rivano, and F. Le Mouël, "A survey of smart parking solutions," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 12, pp. 3229–3253, 2017, doi: 10.1109/TITS.2017.2685143.

[2] D. Shoup, "Cruising for parking," *Transport Policy*, vol. 13, no. 6, pp. 479–486, 2006, doi: 10.1016/j.tranpol.2006.05.005.

[3] S. Du, M. Ibrahim, M. Shehata, and W. Badawy, "Automatic license plate recognition (ALPR): A state-of-the-art review," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 23, no. 2, pp. 311–325, 2013, doi: 10.1109/TCSVT.2012.2203741.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[5] Ultralytics, "YOLO11 documentation," 2024. [Online]. Available: https://docs.ultralytics.com/models/yolo11/

[6] Jaided AI, "EasyOCR: Ready-to-use OCR with 80+ languages," GitHub repository. [Online]. Available: https://github.com/JaidedAI/EasyOCR

[7] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[8] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[9] Espressif Systems, *ESP32 Series Datasheet*, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[10] TowerPro, *SG90 Micro Servo Motor Datasheet*. [Online]. Available: http://www.towerpro.com.tw/product/sg90-7/

[11] S. Ramírez, *FastAPI Documentation*, 2018. [Online]. Available: https://fastapi.tiangolo.com

[12] Vercel, *Next.js Documentation*. [Online]. Available: https://nextjs.org/docs

[13] SQLAlchemy, *SQLAlchemy 2.0 Documentation*. [Online]. Available: https://docs.sqlalchemy.org

[14] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015, doi: 10.17487/RFC7519.

[15] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[16] OWASP Foundation, "Password storage cheat sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[17] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255, doi: 10.1109/CVPR.2009.5206848.

[18] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.